algorithms

THE ALGORITHMS365 REPORT · 2026

What Companies Actually Ask

How India hires freshers in 2026 — the rounds, the questions, and what engineering colleges must change. A research report for principals, heads of department, placement officers and students.

Algorithms365 Technologies LLP, Bengaluru

Prepared for VTU-affiliated and other Indian engineering institutions

November 2026 · 35 employers studied · every claim sourced

BEFORE YOU START

How to read this report

This report answers one question: **when an Indian company interviews a final-year engineering student in 2026, what actually happens in the room, and what is actually asked?**

We studied 35 employers across four segments — IT services, global capability centres, consumer-internet product companies, and mid-tier product firms and startups — and reconstructed their fresher hiring processes round by round. We then built question banks for each round type, organised by subject, with a deliberate bias towards **the things that are not covered by a standard college laboratory**.

The report is long because the subject is. It is designed to be read in parts rather than cover to cover.

If you are a…	Read first
Principal or Director	**Part A** (the shift) and **Part E** (what must change), particularly **E 3** on measuring whether any of your training spend is working. These carry the institutional decisions: curriculum, skill labs, assessment, industry partnership and the credit question.
Head of Department	**Part C** (the rounds), **Part D** (the question banks) and **Part E 1** (the gap analysis against the syllabus). This is where teaching plans are made.
Training & Placement Officer	**Part B** (the employers, round by round), the **eligibility traps** in Appendix 1, and **E 3 and E 4** — the assessment programme and the placement-season playbook.
Student	**Part D** (the question banks) and **E 5** (the semester-by-semester plan). Start with the round your next drive will run.

A note on honesty

Almost no Indian company publishes its interview process. Most of what circulates online is either candidate recollection or search-optimised content with no source at all. We have therefore labelled the evidence behind every process description, and we have left gaps as gaps. Where a figure could not be traced to a real source, this report says so rather than printing it. Section B 0 explains the labelling, **Appendix 3 links the public source behind every question**, and **Appendix 4 sets out what this report does not know**. All three are worth reading before you quote anything from it.

Everything in this report was compiled in the first week of November 2026. Hiring processes change every cycle. Treat the structures here as the shape of the thing, not as a guarantee, and always verify against the specific drive notification your placement cell receives.

EXECUTIVE SUMMARY

Twelve findings

The detail is in the rest of the report. If you read nothing else, read these.

On the market

1. **The industry is growing and the big employers of freshers are shrinking at the same time.** NASSCOM reports the Indian tech industry added about 1.35 lakh net jobs in FY26. Over the same year the combined headcount of TCS, Infosys, Wipro, HCLTech and Tech Mahindra *fell* by 7,389. Both are true. The growth has moved to GCCs, product firms and smaller companies.

2. **One in four entry-level IT jobs disappeared in a year, while demand for tech work rose.** That is foundit's measurement, not a projection: tech occupation demand up 10% year on year, entry-level hiring down 10%, and IT firms' share of all fresher hiring down from 32% to 24%.

3. **GCCs are the single biggest gainer and Bengaluru is their capital.** GCCs rose from 8% to 11% of India's fresher hiring in a year. India now has 2,117 GCCs; Karnataka holds roughly 30% of them and about 32% of their talent. For a VTU college this is the most important employer segment in the country, and it is next door.

4. **But the GCC door is narrowing too.** Freshers are projected to be 16% of GCC hiring in FY27, down from 21%. GCCs buy 78% of their talent externally and build only 22%. They are taking a growing share of a shrinking fresher pie.

On what AI changed

5. **The work has moved from writing code to judging code.** Agent-started pull requests were over 40% of all pull requests on GitHub by September 2026, up from 17% in March. Meanwhile 84% of developers use AI tools but only 33% trust the output, 66% name “almost right, but not quite” as their biggest frustration, and 45% say debugging AI-generated code takes *longer*. The scarce skill is no longer typing. It is verification.

6. **The fresher job has been redefined, not deleted.** The routine coding, test-writing and documentation that used to justify a large entry-level cohort is exactly what AI absorbed first. What is left for a fresher is the part that was previously learned on the job in year one: deciding what to build, reading unfamiliar code, and owning an outcome.

7. **AI skill is already priced.** At 0–3 years' experience, AI roles pay about 20% more than traditional IT roles. AI-led roles are now 4% of fresher hiring, up from about 1% — and 51% of those are AI *operations*, not model building. The accessible opportunity is in using AI well, not in training models.

On what is actually asked

8. **There are nine distinct rounds, and a typical college prepares students for two of them.** Aptitude and DSA are drilled. Machine coding, low-level design, system design, the project cross-examination, written and spoken English production, and CS-fundamentals depth are largely not. Part C sets out all nine.

9. **The machine-coding round is the biggest single gap.** Product companies give a student 60 to 150 minutes to build a working application in memory — no database, no user interface — and mark it on whether it runs, whether it is modular, and whether a new requirement can be added without rewriting it. Nothing in a standard lab record prepares a student for this.

10. **The project round is a cross-examination, not a presentation.** Interviewers ask why you chose that stack, whether it would survive production, how you would improve it, and — in one documented case — how exactly you calculated the "15% improvement" claimed on your resume. One deeply understood project beats five shallow ones.

11. **Core subjects matter more at GCCs than anywhere else, and Salesforce says so officially.** It names data structures and algorithms, object-oriented programming, operating systems, DBMS and computer networks, and recommends college course notes for them. Target's June 2026 interview asked the seven OSI layers, DDL/DML/TCL and transaction management. The VTU syllabus is directly placement-relevant — if it is taught for use rather than for the examination.

12. **Communication is separately gated at five of the nine IT services firms.** Accenture's first round is an AI-graded spoken-English test. Wipro sets a written essay. Tech Mahindra sets a picture-based story. These eliminate candidates before anyone looks at their code.

PART A

The shift

What has actually happened to fresher hiring in India, measured rather than asserted — and what artificial intelligence did to the entry-level engineering job.

- A 1 The state of fresher hiring, 2026
- A 2 What AI changed about the entry-level job
- A 3 Where the growth is — and where it is not

A 1

The state of fresher hiring, 2026

Two national employability numbers circulate in India and they disagree. Both are real; they measure different things. A college should know which is which before quoting either.

Employability: two numbers, two methods

Measure	Figure	What it actually measures
Mercer | Mettl, India's Graduate Skill Index 2025	**42.6%** of graduates employable (2024, down from 44.3% in 2023); technical roles 42.0%	Assessment scores measured against role benchmarks — a skills-readiness measure, not a record of offers. Over 1 million students, 2,700+ campuses, 2,800+ skills. Tier 1 48.4%, Tier 2 46.1%, Tier 3 43.4% — a gap of only about five points between the best and the rest.
India Skills Report 2026 (Wheebox/ETS with CII, AICTE, AIU)	**70.15%** for BE/BTech (down from 71.5%); CS ~80%, IT ~78%; overall 56.35%	The Global Employability Test, taken by 100,000+ self-selected candidates, plus employer surveys. A different threshold and a far smaller, self-selecting sample — which is why it reads roughly 28 points higher.

The finding behind both numbers

Mercer's tier data is the one worth dwelling on. The difference in assessed employability between a Tier 1 and a Tier 3 college is about **five percentage points**. The difference in placement outcomes between them is vastly larger than that. The gap is therefore not mainly a gap in student capability — it is a gap in *demonstrated, evidenced* capability and in employer access. That is a problem a college can act on.

Placement: the most recent defensible figure

AICTE's own data for undergraduate engineering from 2019-20 to 2022-23 records roughly **3.43 million students enrolled and 1.64 million placed — 47.7%**. In other words, about **52% of BTech students had no recorded campus placement**. Two honest caveats: the placement field is self-reported by institutions, and it is measured against enrolled rather than graduating students, so it understates the true rate somewhat. AICTE has not published equivalent data for 2023-24 or 2024-25 in the public domain, which is itself worth noting.

The system is large and growing: **5,903 engineering and technology colleges** with enrolment close to **2.6 million** in the year ended March 2026. Seat fill rates have risen steadily to 75.07% in 2024-25, driven almost entirely by computer science and allied branches. Karnataka enrolled 1,82,519 students in 2024-25 at an 80.47% fill rate, against 1,04,875 at 51.82% in 2020-21.

What the last three seasons looked like

Year	What happened
2024	The trough. An alumni-group compilation reported 35–40% of students unplaced at the older IITs and 50–60% at the newer ones. *Compiled by an alumni group, disputed by institutions — treat as indicative.*
2025	A partial recovery, driven by GCCs. VIT reported around 7,600 students placed via 871 companies, and said **about half of its offers came from GCCs**. Stated campus targets that season: TCS 40,000+, Infosys 20,000, HCLTech 10,000. Entry-level IT services pay sat at **₹3.6–4 lakh**; the GCC average was around **₹6 lakh**.
2026	Volumes flat to down at entry level, with the top of the market pulling away. At the IITs, pre-placement offers rose 10–20% but salaries were broadly static; quantitative-trading firms at IIT Hyderabad were up 50% with 25–40% salary increases, while McKinsey, BCG and Bain did not visit campuses at all.

One thing this report will not tell you

There is **no authoritative national dataset** on median versus top engineering campus packages in India. AICTE does not publish it; NIRF collects institution-level median salary but publishes no national series; the "highest package" figures in the press are individual-college claims, often international CTC converted at list rates. The widening spread is real — entry-level services pay has been close to static since the late 2010s while AI and quantitative roles rose 25–40% in a single season — but we will not print a precise national ratio, because one does not exist.

The clearest instruction in the whole dataset

From foundit's Insights Tracker, July 2026: **nearly six in ten fresher job postings now prefer or require prior internship experience. Only 42% remain open to a candidate with none.** If a placement cell takes one action on the strength of this report, it should be to make sure every student has real, documented work experience before the final-year season — through internships, industry projects or supervised builds. Alongside this: STEM graduates are 42% of fresher hiring, and 15% of fresher postings specify no educational qualification at all.

A 2

What AI changed about the entry-level job

The common claim is that AI is taking fresher jobs. The evidence says something more precise and more useful: AI took the *tasks* that fresher jobs used to be built from, and the job is being rebuilt around what is left.

How much has actually changed

The strongest first-party measurement available is GitHub's own. By **September 2026, agent-started pull requests were over 40% of all pull requests** on GitHub, up from 17% in March 2026 — roughly 110 million agent-started pull requests in that month alone. 55% of GitHub Copilot's monthly active users now use agentic features, up from 35% a year earlier. India is now the second-largest developer population on the platform at **32.0 million developers**, having added more than 10 million in a single year.

We use that figure rather than the widely repeated "AI writes 30% of Google's code" claims, which trace back to chief-executive remarks on earnings calls about accepted characters in specific repositories and do not mean what they are usually taken to mean.

The single most important pairing in this report

From the Stack Overflow Developer Survey 2025, roughly 49,000 respondents: **84% of developers use or plan to use AI tools** and 51% of professionals use them daily. But **only 33% trust the accuracy of AI output** (3.1% highly, 29.6% somewhat) while **46% distrust it**. **66% name "AI solutions that are almost right, but not quite" as their single biggest frustration**, and **45.2% say debugging AI-generated code is more time-consuming**.

Read those together. The machine now produces most of the first draft, and the expensive, scarce, human part of the job is deciding whether the draft is right. That is a judgement skill, and judgement is learned by building things and being wrong about them — which is precisely what a lab record does not provide.

The numbers on Indian IT

Measure	Figure
Industry employment, FY26 (NASSCOM)	59.5 lakh total, up from 58.15 lakh — net growth of only about **2.3%**, with net additions of roughly 1.35 lakh
The five largest IT services firms, FY26	Combined headcount **fell by 7,389**, against a net addition of 12,718 in FY25
TCS fresher intake	**44,000+ in FY26**; **25,000 offers for FY27** as at April 2026 — down from 40,000–42,000 in prior years
Infosys	20,000+ in FY26; **"at least 20,000"** planned for FY27
Wipro	**7,500 in FY26**; **no FY27 target set**, citing demand volatility
Cognizant	**~25,000 freshers planned for 2026** — the outlier holding steady

Measure	Figure
Entry-level IT roles	**"One in four entry-level IT jobs has disappeared over the past year, even as demand for tech work has continued to grow."** — Tarun Sinha, CEO, foundit, August 2026

It is worth being precise about causation. When TCS announced roughly 12,000 job reductions in July 2025, chief executive K Krithivasan explicitly denied AI was the reason: *"This is not because of AI giving some 20 percent productivity gains... This is driven by where there is a skill mismatch, or where we think that we have not been able to deploy someone."* He also noted that freshers need around **nine months of training before project deployment** while lateral hires contribute immediately — which is the real economic pressure acting against large fresher cohorts.

What employers say they now want

BCG X, writing in November 2025, names four skills directly:

- **System design** — understanding *why* a piece of code should exist and how it fits the wider system.
- **Debugging** — learning how systems behave by finding failures, including the gaps in AI-generated code such as weak scalability or missing architectural context.
- **Judgement** — choosing between AI outputs *and being able to explain the choice*.
- **Explaining their work** — saying *why* the code works, not merely that it runs.

BCG's warning phrase is worth quoting to students directly: the risk is teams that **"can write code that runs but can't explain why it works."** The same research found that **38% of engineering leaders agree AI tools have reduced the direct mentoring juniors receive from seniors**, and 43% named critical thinking as the most sought-after skill of the next three years.

> ### Why this lands on colleges
>
> If AI has absorbed the routine work that juniors used to cut their teeth on, and if senior engineers are mentoring juniors less than they used to, then the apprenticeship that used to happen in a graduate's first year on the job now has to happen **somewhere else**. The only institution positioned to provide it before employment is the college. That is an inference rather than a measured finding, but it follows directly from the evidence, and it is the central argument of this report.

Two Indian industry statements worth putting in front of a governing council

NASSCOM, July 2026: AI is beginning to handle routine coding and deep technical skills are slipping; only **23% of professionals are "AI-native"**. NASSCOM's recommendation to colleges is to move beyond basic coding lessons towards **engineering judgement and creative problem-solving**.

Rajesh Nambiar, President, NASSCOM, November 2026: graduates risk arriving with skills that are *"very obsolete already"* if education cannot change fast enough, and *"the level of transformations required in the curriculum has never been this significant."* On the prospects for freshers: *"it is a concern, and I will not sugarcoat it any differently."* *Reported by Elets CIO on 8 November 2026 from a Nikkei Asia interview of 6 November 2026; we have not seen the original.*

Ravi Kumar S, Chief Executive, Cognizant, October 2025, on the shape of the new organisation: *"That pyramid is going to be broader and shorter, and the path to expertise is going to be faster."* And, against the prevailing gloom: *"I think we'll need more school graduates in the AI era... Applying intelligence is the asymmetry."*

A 3

Where the growth is — and where it is not

If the large IT services firms are no longer the automatic destination for a VTU graduate, the practical question is what replaced them. The answer is global capability centres, product companies and startups — each with a different bar.

Global capability centres

Metric (FY26 unless stated)	Figure
GCCs in India	**2,117** companies operating **3,728** units, up 32% since FY2021
People employed	**~2.36 million**
Ecosystem revenue	**US$98.4 billion** — passing the 2030 target roughly four years early
GCCs with embedded AI/ML capability	**Over 1,200**, with 250+ dedicated AI centres of excellence and ~250,000 AI professionals
Share of India's fresher hiring	Rose **8%** → **11%** in a year — the largest gain of any employer category
GCCs engaging with universities	**Over 90%** of leading GCCs; more than half co-innovate with startups

Karnataka's position — the single most relevant fact for a VTU college

Karnataka hosts **875+ GCC units, about 30% of the national total**, with a GCC talent pool of **over 0.6 million — roughly 32% of India's** — and a market value of US$22.2bn, about 34% of the Indian total. Bengaluru takes about **41% of national GCC office leasing**. Karnataka accounts for close to **30% of India's AI-related GCC hiring**, and the state's GCC policy targets **500 new GCCs by 2029**, with an AI Skilling Council and a ₹100 crore innovation fund explicitly intended to support **GCC–academia collaboration**.

A college in Karnataka is sitting inside the largest concentration of this employer type in the country, at the moment when over 90% of those employers say they want to work with universities. That is an opportunity with a deadline on it.

The honest other half of the GCC story

GCCs should not be presented to students as a guaranteed safe harbour. The same body of evidence shows:

- Freshers are projected at **16% of GCC hiring in FY27, down from 21%**. GCCs **buy 78%** of their talent externally and build only 22% — and of those external hires, 41% come from IT services and 32% from other GCCs. They largely recruit experienced people.

- Around **5,500–6,000 roles were cut across Indian GCCs during 2025**, and average headcount per centre has stopped growing (roughly 1,115 per centre in 2026, against about 1,130 in 2024) even as the number of centres rises.
- Zinnov classes **18% of GCC work as commodity tasks and 37% as procedural work — together 55% facing AI displacement pressure**. That 55% is a fair description of the work freshers were historically hired to do.
- **Talent quality is the single biggest constraint GCCs report**, and they cite graduate job-readiness at 42.6%. That sentence is simultaneously the problem and the commercial case for fixing it.

The fair summary for a student: **GCCs are taking a growing share of a shrinking fresher pie, and they are selective about who they take.** Both halves need saying.

Startups

Startup job postings rose from about 116,080 in April 2025 to 130,010 in April 2026, roughly 12% growth — down from 32% growth the previous year. **AI and data roles are now around 32% of startup tech hiring, up from 24%.** But the **entry-level share of startup hiring fell from 41% to 36%**, with 4–10 year professionals taking nearly half. Most startup pay sits in the **₹3–10 lakh** band.

The striking geographic finding: **Tier-2 and Tier-3 cities held 36% of startup jobs in April 2026, up from 9% two years earlier**. Bengaluru remains the largest single location at 20% but its dominance is declining. For colleges outside the metros this is a genuine shift in reach.

The contradiction to present honestly

Both of these are true at once

Demand signals are up. TeamLease EdTech found about 73% of employers planned to hire freshers in January–June 2026, with Bengaluru intent at 84%. Naukri's February 2026 index showed fresher hiring up 17% year on year, AI/ML roles up 49%, and the 20+ LPA fresher band up 30%.

Advertised entry-level share is down. One tracker of 18,752 live Indian tech postings in October 2026 found only 799 — **4.3%** — were entry or junior level, against 51.5% senior: roughly **twelve senior openings for every entry-level one**, worse than the eight-to-one of three months earlier.

The reconciliation is the argument of this report. **The market is improving for freshers who can demonstrate what they can do, and getting harder for those who cannot.** As Shantanu Rooj of TeamLease EdTech put it: *"The real divide today is between candidates who can demonstrate applied skills and those who cannot."* Or, from NASSCOM's own analysts: *"the question is no longer 'Where did you study?' but 'What can you do?'"*

PART B

The employers

Thirty-five companies across four segments, with their fresher hiring processes reconstructed round by round — and an honest account of how much of this is officially published and how much is not.

- B 0 How we sourced this, and how to read the labels
- B 1 The four segments and what each screens for
- B 2 IT services — nine employers
- B 3 Global capability centres — eleven employers
- B 4 Product and consumer internet — eight employers
- B 5 Mid-tier product and startups — seven employers
- B 6 The comparison tables

B 0

How we sourced this

This section exists because the alternative — presenting candidate recollections as company policy — is how students end up preparing for the wrong thing with complete confidence.

The central problem

Almost no Indian company publishes its interview process. Across 35 employers studied, we found genuinely official, current, public process documentation for only a handful: TCS (its All India NQT page is detailed and authoritative), Salesforce (an engineering blog describing its rounds and what it screens for), Cisco (a dated stage calendar for Code with Cisco), Myntra (a fully dated campus pipeline with exact question counts and time limits), and the non-degree academies run by Zoho and Freshworks.

Everything else is reconstructed from candidate accounts. Those are useful — they are often detailed and specific — but each describes **one candidate, in one cycle, for one role**. They are not policy and they go out of date quickly.

The labels used in this report

Label	Meaning and how much weight to give it
OFFICIAL	From the company's own careers page, programme page, engineering blog, press release or quoted earnings commentary. Reliable. Quote freely.
PRESS	Mainstream business media, usually quoting a named executive. Reliable for figures and statements; check the date.
CANDIDATE	Written by an individual candidate — GeeksforGeeks, LeetCode Discuss, Glassdoor, Medium, workat.tech. Indicative of style and structure. **Never present to students as company policy.**
NOT VERIFIED	We looked and could not find a credible source. We have left these blank rather than filling them in.

Four warnings that affect how you should use Part B

1. **A great deal of "2026 interview guide" content online is synthetic.** During this research we found numerous pages carrying 2026 datelines that gave round structures and salary bands with no named author, no named candidate and no traceable source — including one quoting a US-dollar salary band for an India-based fresher role. We excluded them. If your students are preparing from such pages, they are preparing from fiction.
2. **Aggregator pages are often auto-refreshed.** Several carry a 2026 label but were written in 2020–2022 and merely "last updated" since. We have given true dates where we could find them and flagged anything pre-2025 as potentially stale.
3. **Compensation figures are the weakest part of the public record.** Apart from TCS's official NQT offer table and a handful of published internship stipends, essentially no company in this study publishes fresher CTC. Circulating figures trace back to crowd-sourced self-reports. **This report deliberately prints very few salary numbers**, and labels every one. For real figures, a placement cell's own offer letters are the only defensible source.

4. **No company in this study publishes a policy on whether AI tools may be used in its assessments.** We searched specifically for this across all 22 and found nothing official from any Indian employer. The one indirect signal is Infosys running its October 2026 Specialist Programmer coding assessment in person at its own test centres rather than remotely. If this question matters to your institution — and it should — the only way to answer it is to ask each campus-recruitment contact directly.

What this means for a placement cell

Your own drive notifications and campus brochures are **better evidence than anything on the internet**, including this report. The aggregator sites are, in many cases, paraphrasing brochures that your office already holds. Build an internal archive of them: dated, by company, by year. Within three cycles you will have the most accurate picture of fresher hiring available to any institution in your region, and you will be able to correct this report.

B 1

The four segments

The four employer types in this study screen for genuinely different things. A student prepared for one is not automatically prepared for another, and a college that trains for only one will see its results concentrate in that segment.

Segment	What the process looks like	What it is really testing
IT services	A large eliminatory online test — aptitude, verbal, logical reasoning — plus coding or pseudocode, then a technical interview and an HR round. One pipeline, score-banded tracks.	**Trainability and reliability at volume.** Can you follow instructions, reason numerically, write and speak clearly in English, and absorb nine months of training? Communication is separately gated at five of seven.
Global capability centres	An online assessment mixing **CS-fundamentals MCQs with two to four coding problems**, then two or three technical interviews, often a managerial round and usually an explicit HR/values round.	**Depth in the core subjects.** Operating systems, DBMS, computer networks and OOP are examined directly and seriously. This is the segment where the VTU syllabus pays off most literally.
Product and consumer internet	A coding assessment, then DSA interviews, then a **machine-coding or low-level-design round where you build something that runs**, then a hiring-manager round that cross-examines your own project.	**Can you build and defend working software?** Design judgement, code quality, extensibility and ownership. The hardest segment to prepare for from coursework alone.
Mid-tier product and startups	Highly varied: long-form take-home builds, multi-hour programming rounds, hackathons, and project deep-dives. Several explicitly ignore college tier and CGPA.	**Demonstrated craft.** Several of these are the most accessible high-quality employers for a non-elite college — and the least prepared-for.

Two progression architectures inside IT services

This distinction decides whether a student can reach the higher pay bands, and most students do not know it exists.

Model A — one test, score-banded tracks

TCS (Ninja / Digital / Prime), Cognizant (GenC / GenC Pro / GenC Next), Capgemini (Analyst / Senior Analyst), LTIMindtree, Tech Mahindra, Accenture. One application, one pipeline; the higher band is **awarded, not applied for**.

What a student must know: attempt every optional and advanced section. TCS states this officially — the Advanced section of the NQT is **mandatory to be considered for Prime or Digital**. Skip it and you are Ninja-only no matter how well you did on the rest.

Model B — separate route, separate entry

Infosys. Specialist Programmer and Digital Specialist Engineer run as a **distinct drive with its own three-hour, three-problem coding test at an Infosys centre**, with restricted branch eligibility. Power Programmer is entered through top performance in HackWithInfy or advanced InfyTQ levels, bypassing the aptitude test entirely. Wipro Turbo is a variant: **gated by institute and branch** rather than by score.

What a student must know: the highest-paying Infosys roles are **completely invisible** to someone who only sits the campus aptitude test. And if your college is not on Wipro's institute list, Turbo may be structurally unavailable regardless of performance. A placement cell should say this out loud rather than letting students assume a single test decides everything.

No company publishes its cut-offs

Not one employer in this study publishes a numeric score threshold for its higher track. TCS states explicitly that cut-offs vary by hiring cycle, so a score that qualifies for Ninja in one drive may qualify for Digital in another. **Any specific cut-off number circulating online is unverified.** Students calibrate against these numbers and should be told they are fiction.

B 2

IT services

Still the largest single channel for VTU graduates by volume, and the one changing fastest. Their share of fresher hiring fell from 32% to 24% in a year, but the absolute numbers remain large and the top tracks now pay multiples of the base.

TCS

Ninja · Digital · Prime · one test, three outcomes · OFFICIAL the best-documented process in this report

Tracks and pay (from the official All India NQT offer table): **Prime** ₹9.09–9.30 LPA for UG, ₹11.59–11.80 LPA for PG. **Digital** ₹7.09–7.30 LPA UG. **Ninja** a stated minimum of ₹3.40 LPA UG, ₹3.60 LPA PG.

Eligibility: B.Tech/B.E/M.Tech/M.E/MCA/M.Sc/M.S, **any specialisation**. Minimum **60% at 10th, 12th, diploma, graduation and post-graduation**. **No pending backlogs** at selection. Course must finish in the stipulated duration — **extended education is not permitted**. Total academic gap no more than 24 months. Age 18–28.

1 · **TCS NQT, 190 minutes, at a TCS iON centre.** Part A Foundation (75 min): Numerical Ability 25 min, Verbal Ability 25 min, Reasoning Ability 25 min. Part B Advanced (115 min): Advanced Quantitative & Reasoning 25 min, then **Advanced Coding, 90 minutes**.

2 · **Technical interview.** Evaluation focuses on the skill chosen at registration. CANDIDATE reports cover programming fundamentals, data structures, basic DBMS and SQL, and OS concepts; Prime adds review of an AI or data project — its design decisions and whether it actually runs.

3 · **Managerial / HR.** CANDIDATE

The thing to tell every student: the Advanced section is officially mandatory for Prime or Digital consideration. The gap between skipping it and attempting it is the gap between ₹3.4 lakh and ₹9 lakh.

Infosys

Systems Engineer · Digital Specialist Engineer · Specialist Programmer · Power Programmer · separate routes, not one ladder

Pay: PRESS and a live October 2026 drive notification both put the **Specialist Programmer and Digital Specialist Engineer band at ₹7–21 lakh**, with SP L1 reported at ₹10 LPA plus a joining bonus and DSE at ₹6.25 LPA. The long-standing Systems Engineer figure of ₹3.6 LPA is widely reported but we could not verify it officially and it may be out of date.

Eligibility, SP/DSE route: branches restricted to CSE, IS/IT, Data Science, AI/ML, Cybersecurity, Software Engineering, ECE, EEE and related. Note one detail worth telling students: Infosys computes academics as the **simple arithmetic average across all semesters and all subjects, including electives, practicals and languages** — which is not how most universities compute CGPA.

Systems Engineer route: an aptitude-style online assessment, then technical and HR interviews. CANDIDATE sources disagree on the format — one describes three sections (reasoning, mathematical/logical, verbal, 95 min, 65 questions), another five sections adding **pseudocode and puzzles**. Both patterns have been used; which one appears varies by drive and year.

SP/DSE route: a **three-hour coding assessment with three problems** (reported as one easy, one medium, one hard), taken **in person at an Infosys test centre**, then a technical interview walking through the candidate's own solutions, then HR. Hard-problem topics reported: dynamic programming — knapsack, longest common substring, palindromic subsequence.

Power Programmer route: entered via top performance in **HackWithInfy** or advanced **InfyTQ** levels, bypassing the aptitude round entirely.

The structural point: a student who only sits the campus aptitude test can never reach the ₹10–21 lakh bands. Those require applying to a separate drive, or competing in HackWithInfy, in the preceding year.

Wipro

Elite National Talent Hunt · Turbo · WILP · weakest current documentation in this report

Pay CANDIDATE: Elite NTH around **₹3.5 lakh** joining CTC with tenure bonuses at 6, 18, 24 and 36 months. Turbo (2025 batch) **₹5.5 lakh plus a ₹1 lakh retention bonus**. WILP pays a **stipend, not a CTC** — about ₹15,500/month in year one — against a **60-month service agreement**, and leads to a sponsored M.Tech.

Eligibility, Elite NTH: 60% at 10th and 12th, 60% or 6.0 CGPA in graduation, and — notably more lenient than TCS or Infosys — **one backlog is allowed** at the time of assessment. Excluded branches: Fashion Technology, Textile Engineering, Agriculture & Food Technology.

Elite NTH: registration → online assessment (verbal, analytical, quantitative, **written communication**, then coding) → a single combined **Business Discussion** interview → offer. Reported durations conflict between sources (80 minutes versus 128 minutes); treat both as uncertain.

Turbo: online test → **voice assessment** → Business Discussion → pre-skilling.

Two things to flag to students. First, Wipro uniquely tests **written English production** — a scored essay, marked on organisation, diction and ideas, with marks deducted for spelling, grammar and punctuation. Second, **Turbo is restricted to CS, IT and circuit branches at around 46 named institutes**. If your college is not on that list, there is no documented mechanism to be upgraded into Turbo from Elite NTH. Say so plainly rather than letting students assume otherwise.

PRESS Wipro hired **7,500 freshers in FY26** and **declined to set an FY27 target**, citing demand volatility.

Cognizant

GenC · GenC Pro · GenC Next · score-banded, with an extra gate for the top track

Pay CANDIDATE, from 2024–25 campus brochures: GenC (Programmer Analyst Trainee) **₹4 LPA**; GenC Pro **₹5.4 LPA**; GenC Next **₹6.75 LPA**. A fourth name, "GenC Elevate", circulates but does not appear in either brochure — treat it as unconfirmed.

Eligibility: at least **60% at X, XII, diploma, UG and PG with no rounding** — 59.9% does not qualify. No standing arrears. Freshers only; prior work experience is not counted.

Registration: candidates **choose two of three skill clusters — Java, Python or C#** — each paired with ANSI SQL plus web or cloud basics.

1 · **Communication assessment** (online, standalone).

2 · **Aptitude, then a technical assessment** weighted roughly 85% on the chosen language and SQL.

3 · **Automata Fix** — a coding-*debug* round: you complete or fix a partially written or buggy program rather than writing one from scratch. Recurring bug types reported: off-by-one errors, missing return statements, incorrect loop termination.

4 · **Technical interview**, then HR.

Distinctive and widely under-prepared for: Automata Fix. CANDIDATE sources suggest it is the main checkpoint for higher-track consideration — a clean, optimised fix signals GenC Pro or Next capability, while merely passing is enough for standard GenC. Debugging someone else's broken code is a distinct skill from writing your own, and almost no college teaches it deliberately.

PRESS Cognizant plans to hire **around 25,000 freshers in 2026** — the one large services firm holding its volume steady.

HCLTech

TechBee (post-Class XII) · standard B.E./B.Tech · the "elite cadre"

The ₹18–22 lakh story, told accurately. PRESS In January 2026 HCLTech's Chief People Officer Ram Sundararajan said on an earnings call: *"We had shared that our entry-level salaries will have the elite Cadre earning 3X–4X of the regular hires."* The cadre is focused on AI, data engineering, digital engineering, cybersecurity and enterprise technology platforms, at a reported **₹18–22 lakh**.

But no source states how a candidate qualifies. We looked specifically. HCLTech has not published eligibility, selection route or cohort size for the elite cadre. This is the single most-hyped number in Indian fresher hiring and the most likely to be misrepresented to students. Quote the figure and the quote; then say plainly that the route is unpublished.

- **Standard B.E./B.Tech route** CANDIDATE: online test (**60 MCQs, 4 sections, 60 minutes, no negative marking** — quantitative, logical, verbal and a technical section covering data structures, OOP, OS, DBMS and code-output prediction) → technical interview, 30–45 min → HR, 20–30 min. Reported cut-off is 70% *per section*, evaluated independently — a high bar, and unverified.
- **TechBee** is a **Class XII entry programme, not a graduate track**, and it carries a **fee of ₹1.40 lakh** for the IT Services track, with a ₹10,000/month training stipend and around ₹2.50 LPA on joining. Colleges sometimes mistake it for a graduate programme; it is not, and the fee must be disclosed in any material shown to students or parents.

Accenture, Capgemini and the rest

Three patterns worth knowing even where documentation is thin · all CANDIDATE

Accenture runs a six-stage process in which **Round 1 is an AI-graded spoken-English assessment — 63 questions across six sections**, testing pronunciation, fluency and listening accuracy. It is a gate before any aptitude or coding. Then a cognitive-ability test (35–45 min), a technical assessment covering **pseudocode, networking, database basics and security**, a coding round where you choose one problem from a set, a technical interview and an HR round. Tracks: Associate Software Engineer ₹4.5–6.5 LPA and Advanced ASE (grade 11A) ₹6.5–9.0 LPA — with **no documented mechanism** explaining how a candidate is routed to 11A.

Capgemini is the outlier that tests **code reading rather than code writing**. Its online assessment has four components: a **game-based aptitude** section, a **pseudocode** section (trace loops, conditionals and variable assignments to determine output — no IDE, no compiler, no specific language), an English communication section, and the **ADEPT-15** forced-choice behavioural inventory, which is not used for elimination. **There is no separate coding round.** A college that prepares students only on DSA problem-solving will under-prepare them for Capgemini entirely. Tracks: Analyst ₹4.0–4.5 LPA, Senior Analyst ₹6.5–7.5 LPA, the latter awarded automatically above a higher (unpublished) cut-off plus an advanced technical round.

Tech Mahindra sets a **picture-based story-writing round** — a written narrative from a displayed image, within a character limit and time window, marked on grammar, punctuation and logical coherence — and it is **elimination-bearing**. Its ELQ test has three separately-scored sections, each with its own cut-off. **LTIMindtree** runs a six-section online test including two coding challenges in 60 minutes and a 50-question psychometric section.

The pattern across all seven: communication is separately gated

Accenture — AI-graded spoken English as Round 1. **Wipro** — a scored written essay, plus a voice assessment in Turbo and WILP. **Tech Mahindra** — picture-based story writing, eliminating. **HCLTech TechBee** — the Versant AI-graded spoken-English test. **Cognizant** — a standalone communication assessment as the first of three online rounds.

Five of seven. These rounds eliminate candidates **before anyone looks at their code**, and they test *production* of English — speaking and writing — not the comprehension and grammar that school English examinations test. Very few colleges prepare for this deliberately. It is among the cheapest gaps to close in this entire report.

B 3

Global capability centres

The segment growing fastest, concentrated in Bengaluru, and the one where the standard engineering syllabus pays off most directly. GCCs examine operating systems, DBMS, computer networks and OOP seriously — often harder than product companies do.

The signature GCC pattern

Almost every GCC in this study runs an eliminatory online assessment that **mixes CS-fundamentals multiple-choice questions with two to four coding problems**. That hybrid is the GCC fingerprint and it differs from the pure-coding assessments common at consumer-internet companies. Then two or three technical interviews, often a managerial round, and usually an explicit HR or values round.

Salesforce

Futureforce · Software Engineering AMTS · OFFICIAL the clearest published statement of what a GCC wants

Salesforce's own engineering blog states what it screens for, and it is worth reproducing almost in full because it is effectively an endorsement of the standard syllabus:

> *"Core CS fundamentals:* ***data structures and algorithms, object-oriented programming, operating systems, DBMS, and computer networks****"* — plus development skills, familiarity with one's own projects, demonstrated **reasoning through discussion rather than quick answers**, clear communication, honesty and willingness to ask for help. Its recommended preparation includes **college course notes for the fundamentals**.

Process OFFICIAL: application → online assessment, **three coding questions in about 90 minutes**, moderate difficulty → technical interviews on problem-solving, domain knowledge, coding, computer fundamentals and projects → a final round combining technical and behavioural questions. The blog notes the process was **recently shortened to three rounds**.

CANDIDATE An older account adds texture: whiteboarding your own project's system flow, implementing an LRU cache, abstract class versus interface with real-life examples, and later static versus dynamic binding, semaphores and the producer–consumer problem, DBMS locking, networking and cryptography. That candidate's own conclusion: *"don't neglect CS core subjects."*

Target India

Target India Apprenticeship Programme · the most current worked example in this report

Eligibility OFFICIAL: pursuing an engineering degree in **CS, IT or a circuit branch**; meets the CGPA requirement; **no active backlogs**. Bengaluru, Manyata Tech Park.

1 · **Online assessment** — around **35 MCQs on aptitude and core computer science**, plus **2 coding questions, easy to moderate**.

2 · **In-person technical interview.** Only two rounds; no separate HR round reported.

What was actually asked CANDIDATE, interview of 13 June 2026 — the most recent account we found anywhere: the **four pillars of OOP**; the **seven layers of the OSI model**; **basic SQL queries, DDL/DML/TCL commands, and database transactions and transaction management**; implement **bubble sort** and state its time complexity. The remainder was resume and project discussion.

That list is, almost exactly, three VTU theory courses. A student who treated OOP, DBMS and Computer Networks as examination subjects rather than working knowledge would struggle; a student who understood them would walk through this interview.

Cisco

Code with Cisco · open registration, all branches · OFFICIAL calendar published

Eligibility OFFICIAL: **all degree programmes and branches**, based in India, passing out in 2027, **no backlogs**. Registration is individual and **Cisco assigns the teams**. This is one of the genuinely open doors in this report.

2026 calendar OFFICIAL: registration from 1 June → online assessment 25 June (**MCQs plus coding, 90 minutes, remotely proctored, no negative marking**) → code-a-thon from 30 June, top 75 announced → **in-person code-a-thon in Bengaluru, 16–17 July**, travel and stay paid → four virtual interview rounds → Letter of Intent within seven days.

What was asked CANDIDATE, September 2025: the assessment was **42 questions — 40 MCQs plus 2 coding problems**, MCQs covering **OOP, computer networks and operating systems**, coding on **tries plus BFS** and on **graphs**. The technical interview covered network topologies, **FTP active versus passive modes**, **DHCP**; real-time operating systems, **processes versus threads**, multithreading, **starvation, deadlock, critical sections and semaphores**; then primality testing, implementing a linked list and detecting a circular linked list. The **managerial** round asked for a detailed explanation of **DNS** and IP address types.

Note the shape: the coding in the assessment is *harder* than the coding in the interview. Cisco weights fundamentals over algorithmic depth.

Walmart Global Tech India

Sparkplug · OFFICIAL restricted to 35 selected colleges · the hardest DSA in this report

Eligibility OFFICIAL: open to a **select list of 35 colleges** (not published). B.E./B.Tech third year, dual degree fourth year, or M.E./M.Tech first year. **Minimum 7.0 CGPA.** Stipend for the top 100 students: **₹1–1.1 lakh per month** plus return airfare and two weeks' accommodation — one of the few officially published compensation figures in this entire study.

1 · MCQ challenge — 25 questions, one minute each, on **DSA, OOP and design, and CS fundamentals**. Around 2,500 of ~20,000 registrants advanced.

2 · Coding Challenge I — 2 problems, 90 minutes. ~1,500 advanced.

3 · Coding Challenge II — 2 problems, 90 minutes. Cumulative score decides; **top 100 selected**.

For the internship track this candidate reports no technical interview at all — selection was purely by assessment score. And the problems are genuinely hard: package distribution using **disjoint set union**; **articulation points via Tarjan's algorithm**; a grid pattern search; and a divisor-query problem with up to 10^8 queries requiring precomputed divisors and GCD properties. Along with Goldman Sachs, this is the only employer here demanding competitive-programming-grade depth.

SAP Labs India, Qualcomm, Oracle and Micron

Four fundamentals-heavy processes · all CANDIDATE unless noted

SAP Labs runs five rounds, and **Technical Round 2 is entirely CS fundamentals** — DBMS, OOP, system design, computer networks and cloud. DBMS is singled out as the differentiator. The source stresses that a perfect answer is not required: interviewers mark **clear reasoning and a logical attempt**. SAP hires from core computing branches, though the account features an **ECE student hired by building strong core CS knowledge** — worth telling non-CS students.

Qualcomm (2027 Campus Hire, Associate Engineer SW; Hyderabad, Bangalore, Chennai, Noida) runs a **60-MCQ, 90-minute test in three 20-question sections**: aptitude and logical reasoning; general programming in **C and OOP**, including **number-system conversion**; and a dedicated **CS-core section on operating systems, computer networks and OOP**. Interview DSA is foundational — what a BST is and how to check whether a binary tree is one, doubly linked lists and their real-world uses, stacks, hashing, graphs, reversing an array in place.

Oracle is the clearest case of **DBMS as the centre of gravity**. A 2024 campus drive ran a 36-question aptitude test containing just **one coding problem** alongside reading comprehension, bar-graph interpretation and OS/DBMS questions. The interviews then went deep: Spring Boot, APIs, dependency injection; cloud basics; **Linux boot process**; and a second round built around a **normalization plus self-join** problem, a **SQL COUNT over a UNION of joins**, classic puzzles, and **dense versus sparse indexing** and SQL-versus-NoSQL scaling. 200+ took the test, 20 were shortlisted, 5 were selected.

Micron runs the strongest OS and DBMS block we found: a four-section written test, then a technical interview with a **dedicated fifteen-minute OOP segment**, then **thrashing, paging and paging algorithms, semaphores versus mutex, zombie processes and RAID**, then **ACID properties, keys, table relationships and the Nth-highest-salary SQL query** — plus puzzles. Its DSA was a basic string character-frequency count.

Arista Networks — and the fresher system-design round

The clearest documented case of a fresher facing a real design round · CANDIDATE

An on-campus fresher process with three rounds: an assessment (**15 MCQs in 15 minutes on computer networks, DBMS, operating systems and DSA**, then an easy coding question in 20 minutes and a medium one in 25); a DSA interview requiring **working code that passes the sample tests** (convert a binary linked list to its decimal value; design a stack returning its minimum in O(1) time and space); and then:

> **A 50-minute system-design round:** design a **URL shortener** that creates short links, **deletes them after a set period**, and **handles concurrent reads and writes in a multiprocessor environment** — followed by basic C questions including *"how would you implement a C++ STL vector in C?"*

Arista's weighting on **computer networks is uniquely high**: TCP/IP, OSI, routing, switching and SDN, alongside OS, SQL and low-level C. If your college has strong CN teaching, this family of infrastructure GCCs is where it converts into offers.

JPMorgan Chase and Goldman Sachs — the two outliers

Neither fits the GCC fundamentals pattern · CANDIDATE

JPMorgan Chase runs the **Software Engineer Program** with **Code for Good** as its final selection round, and its DSA bar is deliberately light. The sequence: a **90-minute HackerRank test with 2 easy-to-medium problems** (merge intervals, substring counting) → a **HireVue recorded video interview** — 5–10 minutes, two pre-recorded behavioural questions, about a minute to think, two attempts each — **which is an elimination round** and which candidates who passed the coding test have failed → a **24-hour in-person hackathon** where NGOs present real problem statements, teams of six or seven build a solution, and **two mentors per team evaluate individual performance**. Both documented candidates note that **winning does not guarantee an offer and losing does not rule you out**; both lost and both were hired. One cycle's test required CGPA 8.5.

Goldman Sachs weights DSA plus **quantitative reasoning** rather than CS fundamentals. Its documented question set runs from convex hull and Trie design to grid path counting and BFS-minimum-operations — but also probability (dice problems, circular-field grazing), expectation, covariance and correlation, matrices and determinants, and conceptual data-science questions. It runs a **CoderPad round where the interviewer adds custom test cases** live. Neither OS nor DBMS was reported in either account.

B 4

Product and consumer internet

The segment that pays most and prepares students least. The defining feature is a round in which you build something that runs — and then defend every decision in it.

Flipkart

GRiD · Girls Wanna Code · WiRED · Vidyarthini · OFFICIAL "open to all specializations, branches, and CGPAs"

Flipkart's own early-careers page states that applications are **open to all specialisations, branches and CGPAs**. Aggregator sites claim a 60% / 6.5 GPA floor; the official line should be the one quoted to students. Fresher roles listed: Software Development Engineer, User Interface Engineer, Data Engineer, Site Reliability Engineer.

GRiD, its flagship contest route, drew **160,959 registrations** in its seventh edition (June–July 2025): a **90-minute, three-question proctored coding test**, a case-study round, then **national finals at Flipkart's Bengaluru headquarters** where the top 48 students in teams of four solve a live problem in 24 hours and present to Flipkart leadership. Finalists are considered for interviews and past winners have joined as SDE-1s.

1 · **Machine coding, 90 minutes** — build a working application.

2 · **Machine coding review, ~45 minutes** — interviewers read your code and ask you to explain the approach. **This two-stage "write it, then defend it" structure is distinctive to Flipkart.**

3 · **Problem solving and data structures, ~1 hour, 3 questions.** Reported: count contiguous subarrays summing to k; check whether a graph is bipartite; k most frequent numbers using a hash map and a size-k min-heap.

4 · **Hiring manager, ~45 minutes** — past projects, technologies, how you stay current. **No system-design round.**

A documented machine-coding problem: design and implement a **COVID vaccination centre system with nine stated requirements**, in 90 minutes, then run the code and explain it. CANDIDATE

PRESS Flipkart's early-talent leadership described fresher intake as likely **stable rather than growing**, and noted explicitly that tier-3 hiring is cheaper — "two or three graduates for the cost of one top-institute engineer". Candidates complete **simulations based on real challenges** such as delivery-route optimisation and supply-chain puzzles.

Swiggy

Associate Software Development Engineer · the best-evidenced current machine-coding loop in this report

Two independent candidate accounts from **December 2025 and March 2026** — unusually current — describe the same four-round shape.

1 · **Online assessment** — 3 problems. One account: sliding window, shortest path in a graph, an array problem involving nth roots. The other: two medium and one hard, with the OA restricted to **Java and Golang only**.

2 · **DSA plus language fundamentals, 1 hour** — two medium problems, then roughly 25 minutes of *verbal* Java: **JDK versus JRE versus JVM, garbage collection, dependency injection, Collections, multithreading**, Spring Boot annotations.

3 · **LLD / machine coding, 1 hour** — the core round. First 10–15 minutes on the candidate's own projects and design choices, then build.

4 · **Hiring manager** — a deep dive on the current project including **a production incident and how it was handled**, SQL and databases, self-rating on the stack, and in one case an unbounded knapsack problem. **No separate HR round.**

Documented machine-coding problems at Swiggy

Shopping cart with multiple discount types — add items, payment, several discount kinds. The successful candidate clarified requirements and assumptions first, then applied the **Strategy pattern** for discounts. *December 2025*

Driver-to-order assignment system — orders are VIP or non-VIP, drivers have ratings, and matching depends on several conditions. Deliverables: design approach, **data-structure choices**, code, and a **dry run**. The interviewer specifically probed the data-structure decisions. *March 2026*

Also reported against Swiggy: Snake and Ladder, order dispatch, restaurant-menu catalogue, Splitwise, parking lot.

A concrete curriculum signal: Swiggy expects **Java and Spring Boot depth** — JVM internals, garbage collection, dependency injection, Collections, multithreading — not just algorithmic ability. A college teaching Java as syntax rather than as a platform leaves students exposed here.

Zomato / Eternal

The counter-example: no machine coding, no DSA grind — architecture first

The single most useful account in this report for a non-elite college: a **final-year CSE student from a Tier-3 university**, off-campus via referral, **December 2025, cleared**. There was **no online assessment, no machine-coding round and no separate HR round**.

1 · **Technical, conversational.** A deep dive on the candidate's **backend projects** — design choices, scaling, modularity. Then a **system design for online ordering**: user flow, restaurant-manager flow, **API payloads**, backend steps, **database schema**. SQL versus NoSQL trade-offs. **Geo queries for nearby restaurants** — the candidate handled latitude/longitude and bounding boxes but was weak on geo-indexing. Many-to-many manager-to-restaurant mapping via a junction table. Service and controller layering. On image uploads the candidate proposed base64; **the interviewer expected pre-signed URLs**.

2 · **Onsite technical, harder.** Redesign your own internship backend and say what improved. Microservices versus monolith. Message brokers — RabbitMQ, Redis Streams, Kafka. **Rate limiter design including a sliding window.** Real-time support chat design. **MongoDB indexing, compound indexes and the left-prefix rule.** One DSA problem. Handling external outages and fallbacks.

3 · **VP of Technology** — culture, ownership, work style. **Offer extended in this round.**

This is the clearest proof in the report that a “DSA plus machine coding” curriculum is not universally sufficient. Zomato hired on **backend architecture judgement and depth in the student's own work**.

Meesho, Groww and Razorpay — three shapes of the build round

All CANDIDATE

Meesho (on-campus 2025) runs four eliminatory rounds: an OA of three problems (easy array; a graph route-optimisation problem; a hard backtracking/DP problem); a DSA round giving **30 minutes to write and explain pseudocode** then five minutes of complexity cross-questions; then an LLD round — **a ride-sharing system with add, find, book and cancel, where function names and parameters are supplied and the candidate fills in the logic**, one hour, with **one test case that must pass**, and the remaining 30 minutes on **OOP basics, SOLID principles and OS concepts — mutexes and race conditions**; then a hiring-manager round interrogating resume projects.

Groww runs five rounds with **no online assessment**, every one eliminatory, including **two separate design rounds**:

> ### Groww, round 2 — machine coding, 90 minutes, written in a shared Google Doc
>
> Design and code a **group chat system**: admins can create up to 5 groups, users can join up to 20 groups, admins can add or remove members. **The code had to run**, and the candidate discussed the design and tested it.

> ### Groww, round 3 — low-level design, 60 minutes
>
> Design a **rate limiter for an image-download service with subscription tiers**: premium users 100 downloads per minute, general users 10 per minute, and excess general-user requests are **queued until the next window rather than rejected**.

Razorpay has the longest documented build round of any company here: a **2.5-hour machine-coding round in which the candidate implements an in-memory database** meeting stated constraints, demonstrated over screen-share with interviewer-supplied test cases. The candidate called it “the most important and most tiring round” and advised completing and double-checking one module before starting the next. A second account instead shows a **90-minute LLD round — design a photo-sharing app with six features plus one bonus** — centred on the **MVC and Singleton** patterns. Both accounts show the hiring-manager round probing resume projects, trade-off arguments and “why Razorpay”.

PhonePe and Uber

Where the campus loop and the experienced loop diverge · CANDIDATE

PhonePe's 2025 campus loop is notably DSA-heavy and contains no machine-coding round. The assessment had four questions — two dynamic programming, one trees, one digit-DP string problem; the candidate **fully solved two, partially solved two, and was still shortlisted**, which is a useful and concrete bar to give students. Then a technical round with **Burning Tree** and **Shortest Path to Get All Keys** — with **full code written on paper, then dry-run** — a hiring-manager discussion, and a second technical round with a Codeforces problem and a custom bitmask-DP problem, again on paper.

For experienced hires PhonePe does run a **90-minute machine-coding round** (build an **activity tag logging system** supporting hierarchical tags and counting tag usage across entities), a DSA round, and a **design round covering both LLD and HLD — "Design Twitter"**. Worth showing students as the bar they are growing into.

Uber runs an **OOP / low-level design round of 50–60 minutes** rather than a scalability round. The documented problem: **design an application controlling multiple devices — a fan, a TV — with four functions, common commands such as ON/OFF plus device-specific ones**. The requirements were **deliberately vague and the candidate asked many clarifying questions**; they began with interfaces then switched to a base `Device` class with inheritance. Another account reports **"design MS Excel"** as the LLD problem. Uber's DSA is medium-to-hard: graphs with state, interval DP, capital/profit greedy with a heap, matrix search, and optimising a string search for an array of a million strings.

What the project round actually probes — consolidated across eight companies

This is the most transferable finding in Part B. Across Flipkart, Swiggy, Zomato, Meesho, Razorpay, CRED, Zepto and Navi, the project round is an **engineering-judgement cross-examination of the student's own work**, not a presentation. Documented questions include: **why this approach and why this tech stack**; **would it work in production and how would you improve it**; the architecture of each component and its role; **redesign your own project and name what improved**; **a production incident you handled**; and — at Zepto — **how exactly did you calculate the "15% cost reduction" claimed on your resume**.

The implication for students is direct and should be repeated until it sticks: **one project you understand completely beats five you can only describe.**

B 5

Mid-tier product and startups

The most accessible high-quality employers for a college outside the top tier — and the least prepared-for. Several of them explicitly do not filter on institution or marks. What they filter on instead is whether you can build something.

Zoho — the case study that matters most to a VTU college

Hires on demonstrated programming ability rather than pedigree, and says so on the record

PRESS Founder Sridhar Vembu, December 2025: *"At Zoho, no job requires a college degree"* — adding that if a manager posts a job requiring one, HR asks for the requirement to be removed. Reporting on Zoho's Tenkasi operation, which employs about 1,200 people, notes that Zoho **recruits from schools and colleges within the district and does not target elite institutions**, hiring from second- and third-tier colleges, rural government schools and polytechnics. Rakeeb Mohamed Mubeen, who leads the Tenkasi developers: *"We don't look at board exam marks or competitive tests."* Vembu: *"The talent is already there. We just have to nurture it."*

The degree-route hiring process CANDIDATE — five to seven rounds, varying by drive:

1 · **Aptitude and logical reasoning** — around 90 minutes, 25 MCQs. Some drives run this offline on paper with **no multiple-choice options at all**.

2 · **Basic programming** — 60 minutes, roughly 10 objective and **output-prediction** questions on C/C++ fundamentals and basic SQL.

3 · **Advanced programming** — the decisive round. Reported as **5–8 problems coded from scratch in C or C++ only** (**Java is not accepted**), scored on **code quality, modularity, boundary and edge-case handling, and efficiency** — not merely on test cases passed. One documented Level-2 problem is a **booking system** with availability checking, cancellation and chart preparation — a miniature application build rather than an algorithmic puzzle.

4 · **Technical interview** — reported to be largely **about the programs the candidate wrote in the previous round**, plus data structures, OOP, DBMS, OS and projects.

5 · **HR interview.**

Two honest caveats before you quote Zoho to students

On the CGPA question: two prep sources directly contradict each other — one states Zoho publishes no CGPA criterion, the other states 60% / 6.5 throughout. Neither is official. The accurate statement is: **Zoho does not publish a CGPA criterion, but individual drive notifications have carried one. Check the specific notification.**

On "Zoho tests building, not DSA": the candidate evidence supports it — working code required, marked on quality and modularity, application-style problems, interview built around your own programs — and no Zoho source contradicts it. But **Zoho has never said this officially**, and DSA does appear in the written and interview rounds. Present it as what the evidence shows, not as a company position.

Zoho Schools of Learning OFFICIAL is a genuine post-Class-XII alternative: a **two-year programme including a one-year internship, no tuition fee, with a stipend**, whose graduates now number **over 2,000 — more than 10% of Zoho's workforce**. Campuses at Chennai, Tenkasi, Tharuvai and Kumbakonam. Rajendran Dandapani, Director of Technology: *"Our commitment to valuing merit over credentials continues to prove that genuine talent flourishes."* It is real but narrow — a few hundred seats a year, concentrated in Tamil Nadu — and should be presented to students as a pathway, not a substitute for degree planning.

BrowserStack — the clearest two-machine-coding-round model

CANDIDATE every round eliminatory

A fresher loop with **two consecutive build rounds**, both explicitly marked on **code quality, testability, modularity and corner cases**:

> ### Machine Coding 1
>
> Build a **`tail -f`-style log watcher**: a server that streams updates from a remote log file, plus a web client that updates in real time.

> ### Machine Coding 2
>
> Build a **stateless web service** that starts, stops, cleans up and reports the active URL for Chrome and Firefox, **without Selenium-based dependencies**. An OS-independent implementation earns bonus points.

Then two technical rounds — with an engineering manager (an array problem in O(n) time and O(1) space; a data-structure design question; **counting word occurrences in a 1 TB file with 8 GB of RAM**) and with a director of engineering (**a project that failed**, HTTP versus HTTPS, status codes, the User-Agent header, production deployment, logging for monitoring, database scaling) — and an HR round that asks about **GitHub use and competitive programming**.

This is the single cleanest example of the thing a student cannot prepare for by practising algorithm puzzles. Both tasks are *build a working tool*.

Juspay — the hackathon as a hiring round

Juspay Hiring Challenge · open to all branches, no stated tier or CGPA filter

One of the few employers here with a published stipend and conversion band CANDIDATE: an 8–12 month full-time in-office internship at **₹40,000 per month**, converting to a full-time CTC reported at **₹21–27 lakh** based on internship performance.

Round 1.1 — 30 MCQs in 30 minutes, taken any time within a 24-hour window.

Round 1.2 — MCQs on systems, algorithms and data structures, logical reasoning, and **physics and mathematics**.

Round 2 — 3 coding problems in about 90 minutes, scored on **test cases passed, time taken and code quality**.

Round 3 — interviews. In earlier editions this stage was a **two-day hackathon in which participants build a fully functional web or mobile application**; the format has changed between editions.

Juspay's direct route additionally reports **LLD, distributed systems and payment-system architecture** in a later technical round, and — uniquely in this study — **functional programming (Haskell and related) for backend roles**. Documented problems include designing an LRU cache and indenting a JSON object.

Zeta, Atlan, Freshworks and HackerRank

Four different answers to "how do we find out if this person can build?"

Zeta (SDE intern, February 2025) runs a **design-then-implement-then-debug** round that is worth describing in full, because it is an unusually complete test of engineering ability: after a two-hour coding assessment (including a **disjoint set union** problem) and a DSA interview that also covered **low-level design of a library management system written in a Google Doc**, DBMS normalization, and **what happens when a URL is entered in a browser**, the final round asked the candidate about **board games they had played**, then to produce the **low-level design of Ludo in a Google Doc — rules, design flow, constraints — then write the code for it, then find and fix bugs in their own code**, then discuss the design's limitations and improvements.

Atlan is the clearest 2025–26 signal on AI-native hiring. Its AI Engineering Internship listing — six months, remote, **₹50,000–60,000 per month**, open to 2026–2028 batches with **no college-tier or CGPA filter stated** — says the company is *"especially excited about **AI-native builders**"*: people who **already use AI tools in their development work**. Selection is online test → **take-home challenge** → interviews. **Public coding contributions are named as a positive signal.**

Freshworks adds, on the campus path, a **40-minute written test of two problems on OOP and linked lists** between the online round and the interviews, and one campus account reports a **take-home assignment: build a file-based key-value data store**. Its **Freshworks Software Academy** OFFICIAL mirrors Zoho's model — twelve months, ₹10,000/month, laptops provided, no fee, for 17–20 year olds from households below ₹20,000/month who have not continued to a degree.

HackerRank is the strongest documented case of the **project deep-dive as the decisive round**. Its intern loop: a **one-hour, one-question coding assessment on basic string processing, explicitly not DSA**; resume shortlisting in which **the CTO reviewed each resume personally with no applicant-tracking system**; a 20-minute CTO conversation with no core technical questions; and then the main technical round — **30 minutes in which the candidate shared their screen and walked through open-source pull requests from a previous internship**, answering questions about that code. The candidate's own conclusion: know how every part of your own work works.

The employers that explicitly ignore pedigree

Zoho — founder on record; hires from tier-2 and tier-3 colleges, rural government schools and polytechnics. **Juspay** — Hiring Challenge open to all branches with no stated tier or CGPA requirement. **Atlan** — no tier or CGPA filter in its listing; names public coding contributions as a signal. **Flipkart** — officially open to all specialisations, branches and CGPAs. **Cisco** — officially open to all degree programmes and branches.

Counter-example, stated plainly so this is not oversold: **Chargebee's** reported criteria are 60% or 6.5 CGPA throughout with no active backlogs. Not every product company is marks-blind.

For a VTU college, these five open doors are worth more institutional attention than a tier-1-restricted programme that will never visit your campus. **Off-campus and pool-campus routes are the realistic entry points**, and they require students to apply individually, early, and without waiting for the placement cell to arrange it.

B 6

The comparison tables

Everything in Part B, condensed. Print these two pages and put them on the placement-cell wall.

Which companies run a build round for freshers?

Company	Build round	Duration	Design round	Evidence currency
Flipkart	**Yes** + a separate code-review round	90 min + 45 min	Folded in	2021–22 *stale*
Swiggy	**Yes** ("LLD / machine coding")	60–90 min	Same round	**2025–26**
Razorpay	**Yes** — longest documented	150 min	Or a 90-min LLD	2021–24 *stale*
Groww	**Yes**	90 min	**Yes** — a second, separate 60-min LLD	undated
Meesho	**Yes** — signatures supplied	60 min + 30 min theory	Same round	**2025**
CRED	**Yes** — helpers supplied, unit tests required	150 min	Separate round	2022 *stale*
BrowserStack	**Yes — two consecutive build rounds**	not stated	—	pre-2025 *stale*
Uber	Not as a named round	—	**Yes** — OOP/LLD, 50–60 min	2022–23 *stale*
Zeta	Design → code → debug	within the round	**Yes**	**2025**
Zoho	**Yes** — advanced programming	75 min	—	**2026**
Zomato / Eternal	**No**	—	**Yes** — architecture is the core	**Dec 2025**
PhonePe (campus)	**No** — DSA-heavy	—	Not for campus	**2025**

Company	Build round	Duration	Design round	Evidence currency
Navi	**No**	—	**No**	2023 *stale*
Arista	Live coding, working code required	45 min	**Yes — 50-min system design**	2022 *stale*
JPMorgan	**24-hour team hackathon**	24 hours	Implicit in the build	2024
Cisco	**Two-day team code-a-thon**	2 days	In managerial round	**2026 calendar**
GCCs generally	No round branded “machine coding”	—	Arista, SAP, Rakuten, Adobe	mixed
IT services	**None** — coding tests only	—	**None**	**2026**

DSA difficulty, by employer

Level	Employers and signature topics
Competitive-programming grade	**Walmart** (Tarjan's articulation points, DSU, number theory at 10^8 queries), **Goldman Sachs** (convex hull, Trie design, broad DP, plus probability and statistics), **CRED** (one medium and two hard in a 90-minute assessment — the hardest single assessment here), **Sprinklr** (three medium-hard in 90 minutes, all expected solved)
Medium to hard	**PhonePe** (digit DP, bitmask DP, burning tree, BFS with state), **Uber** (graphs with state, interval DP, greedy with heap), **Zomato/Eternal** (B+ trees, BST operations, graph plus DP)
Medium	**Flipkart** (sliding window, bipartite graphs, top-k heaps), **Swiggy** (sliding window, shortest path, merge intervals, knapsack), **Groww** (linked lists, decode-ways DP, two-heap), **Cisco**, **Arista**, **Salesforce**, **Adobe**, **Intuit**
Light — fundamentals weighted instead	**Target** (bubble sort), **Oracle** (one coding problem in the whole assessment), **Micron** (character frequency), **Lowe's** (find duplicates), **JPMorgan** (merge intervals), **Qualcomm** (BST validation, reverse an array), **Zepto** (binary search, constructive strings)
Aptitude and pseudocode, not DSA	**Capgemini** (pseudocode tracing only — no coding round at all), **Cognizant** (Automata Fix — debug, don't write), **Accenture**, **Tech Mahindra**, **Wipro**, **TCS Foundation**

The teaching implication, stated precisely

A student who masters **arrays, strings, hashing, linked lists, stacks, trees, basic graphs and basic dynamic programming**, and who **genuinely knows operating systems, DBMS, computer networks and OOP**, clears the assessment stage at the **large majority** of employers in this report. Only Walmart, Goldman Sachs, CRED and Sprinklr demand competitive-programming depth.

This matters because it contradicts the prevailing campus belief that placement preparation means grinding 500 LeetCode problems. For most of this list, **depth in the core subjects plus the ability to build and defend one working system is a better investment** than the next hundred algorithm problems. The exception is real but narrow.

Where CS fundamentals are examined seriously

Directly documented: Target (OSI layers, DDL/DML/TCL, transactions), Cisco (OS, CN and OOP in both assessment and interview), Qualcomm (a dedicated assessment section), Micron (thrashing, paging, semaphores versus mutex, zombie processes, RAID, ACID), Intuit (paging, process states, FCFS and Round Robin, normalization), Oracle (normalization, self-joins, dense versus sparse indexing), SAP Labs (an entire dedicated round), Arista (TCP/IP, OSI, routing, switching, SDN and low-level C), Lowe's, Rakuten (DBMS locking, concurrency, Java internals), Walmart, and **Salesforce — which states it officially**.

Counter-examples, stated honestly: Goldman Sachs weights DSA and quantitative reasoning instead; JPMorgan runs easy DSA, a behavioural video and a hackathon with no fundamentals round reported. Adobe is mixed.

PART C

The nine rounds

Every distinct round type a fresher faces in India in 2026, what each one is actually measuring, and how it is scored — with an honest count of how many of them a typical college prepares students for.

C 1

The nine rounds, decoded

Across 35 employers, every fresher round we documented falls into one of nine types. A college that prepares students for rounds 1 and 2 only — which is the common case — is preparing them for roughly a quarter of what they will face.

#	Round	What it actually measures	Typical format
1	**Screening assessment**	Numerical and logical reasoning, English comprehension, and — increasingly — **code reading rather than code writing**	60–190 min online or at a test centre; MCQs, sometimes pseudocode tracing or debugging; frequently sectional cut-offs
2	**Coding assessment (DSA)**	Whether you can turn a stated problem into correct, reasonably efficient code under time pressure	2–4 problems in 45–120 min; a bar of roughly "two fully solved" is common
3	**DSA interview**	How you **think out loud**: clarify, choose an approach, state complexity, dry-run, then optimise under pressure	45–60 min, 1–3 problems. Sometimes **on paper** (PhonePe, Navi) or in a shared document
4	**OOP / class design**	Whether you can turn a **business description** into classes, relationships and responsibilities	30–60 min; often verbal or on a shared document; SOLID and design patterns expected by name
5	**Machine coding**	Whether you can **build a working application** — in memory, no database, no UI — that is modular, handles edge cases and can absorb a new requirement	**60–150 min.** Code must run. Often followed by a separate review round where you defend it
6	**System design (HLD)**	Whether you can size a system, choose components and justify the cost of your choices	45–90 min. Less common for freshers, but documented at Arista, SAP, Rakuten, Zomato, Zepto and Adobe
7	**Project cross-examination**	Whether the work on your resume is **actually yours**, and whether you understand it deeply enough to defend and improve it	30–60 min with a hiring manager. The most consistently present round across all four segments

#	Round	What it actually measures	Typical format
8	**CS fundamentals**	Operating systems, DBMS, computer networks, OOP and language internals — examined for working understanding, not definitions	MCQs in the assessment, then direct questioning in interview. Heaviest at GCCs
9	**Communication and behavioural**	**Production** of spoken and written English, plus structured answers about conflict, failure and ownership	AI-graded speech, written essays, recorded video interviews, HR conversation. **Eliminating** at five of seven IT services firms

Which rounds does a typical college actually prepare students for?

#	Round	Usually prepared?	Where the preparation comes from, if it happens at all
1	**Screening assessment**	**Yes**	Aptitude training, usually bought in for the final year
2	**Coding assessment**	**Yes**	Online judges, coding clubs, self-study
3	**DSA interview**	**Partly**	Students practise solving but rarely practise *explaining while solving*, which is what is marked
4	**OOP / class design**	**No**	The OOP course teaches the four pillars as definitions; nobody asks students to model a real business
5	**Machine coding**	**No**	Nothing in a standard lab record resembles this
6	**System design**	**No**	Software engineering is taught as lifecycle theory, not as capacity and cost
7	**Project cross-examination**	**No**	Projects are assessed by report and viva, not by adversarial technical questioning
8	**CS fundamentals**	**Taught, not activated**	The syllabus covers all of it; students learn it for the examination and cannot use it in conversation six months later
9	**Communication**	**Partly**	Soft-skills sessions exist; graded production of spoken and written English under time pressure generally does not

The honest scoreboard

Two rounds are properly prepared for. Two are partly prepared for. **Four are not prepared for at all**, and one — CS fundamentals — is taught thoroughly and then allowed to go cold before it is needed.

That is the gap this report exists to describe. It is not a gap in the syllabus's content. Almost everything in rounds 4 to 8 builds on subjects VTU already teaches: OOP, DBMS, Computer Networks, Operating Systems, Software Engineering. The gap is between **knowing a subject** and **being able to use it in front of a stranger, under time pressure, on a problem you have not seen.**

PART D

What is actually asked

Question banks for each of the nine rounds, organised by subject, with a deliberate bias towards what a standard college laboratory does not cover.

- D 1 The screening assessment — aptitude, pseudocode, debugging
- D 2 Data structures and algorithms, by topic
- D 3 Object-oriented design for business scenarios
- D 4 Machine coding — build it in memory, make it run
- D 5 Low-level design
- D 6 System design, with scale and cost
- D 7 The project cross-examination
- D 8 Computer science fundamentals
- D 9 Communication and behavioural

BEFORE PART D

How to use the question banks

Two different kinds of question appear in the pages that follow, and the difference matters.

Marking	What it means
Asked at Swiggy, Dec 2025	This specific question is **documented in a candidate account** against that company and that approximate date. It genuinely came up, and **Appendix 3 gives the public link for every company in this report** so you can open the account and find it. It is still one candidate's recollection of one interview, so treat it as evidence of the *kind* of thing asked, not as a question bank to memorise.
representative	Written by us to match the **documented pattern** for that round and topic, and attributed to nobody. No company asked this exact question as far as we know. These exist so that a department has enough material to teach and set assessments with, rather than four real questions and nothing else.

The one thing not to do with these banks

Do not turn them into a list to be memorised and reproduced. Every round in Part C is scored on **process**, not on recall: whether you clarified the requirement, why you chose that structure, what the complexity is, what happens at the boundary, and what you would change. A student who has memorised fifty solutions and cannot explain one of them performs worse than a student who has genuinely solved fifteen.

The value of these banks is as **raw material for practice under conditions** — timed, out loud, with someone asking why.

D 1

The screening assessment

The round that eliminates the most students, and the one most often prepared for with the wrong material. Only part of it is aptitude. The rest is code reading, code repair, written English and personality inventories.

1 · Quantitative aptitude — the documented topic list

Compiled from the published section descriptions of TCS, Infosys, Cognizant, HCLTech, Capgemini and Juspay. There is nothing surprising here; the point is that the list is **finite and closed**, and a student who works through it systematically in the third year will not need to revisit it.

- Percentages; profit and loss; simple and compound interest; ratio and proportion; averages; alligation and mixtures.
- Time and work; pipes and cisterns; time, speed and distance; boats and streams; trains.
- Number systems; LCM and HCF; divisibility; remainders; **number-base conversion (binary to hexadecimal)** — specifically documented at Qualcomm.
- Permutations and combinations; probability; set theory and Venn diagrams; mensuration; cryptarithmetic.
- Data interpretation: bar charts, tables, multi-variable DI, **data sufficiency**.
- Adobe adds, distinctively: **conditional probability, Bayes' theorem, matrices and determinants**. Goldman Sachs adds expectation, covariance and correlation. Juspay adds **physics**.

2 · Logical reasoning

- Number and letter series; coding-decoding; blood relations; direction sense; analogies; odd one out.
- Syllogisms; coded inequalities; critical reasoning and inference; logical deduction.
- Linear and circular seating arrangements; grid and scheduling puzzles.
- Classic interview puzzles, which appear in *interviews* as well as tests: the **eight balls problem**, the **daughters' ages puzzle**, three ants on a triangle, the faulty-ball weighing puzzle, days of the month using two dice.
  Documented at Oracle, Micron, Adobe, Infosys, Juspay

3 · Pseudocode tracing — the component most colleges miss entirely

At **Capgemini this is the only code-related component in the entire assessment**. There is no coding round. It also appears in Infosys's five-section pattern and in Accenture's technical assessment. The task is to **read code and determine its output**, with no compiler and no specific language.

- Trace a nested loop with a compound update and state the final value of a variable. *representative*
- Trace a recursive function with two base cases and state the number of times it is called. *representative*
- Determine the output when a variable is modified inside a conditional that never executes. *representative*
- Trace pass-by-value versus pass-by-reference behaviour across a function boundary. *representative*
- Trace an array index manipulated inside a loop whose bound is itself modified. *representative*

> ### Why this is worth a dedicated teaching session
>
> Writing code and reading code are different skills, and students practise only the first. A student who can write a correct loop frequently cannot say what an unfamiliar loop will print, because they have never had to simulate execution on paper. This is also, not coincidentally, **the exact skill the AI era rewards** — reading code you did not write and deciding whether it is correct. Pseudocode tracing is the cheapest possible proxy for it, and it takes about six hours of teaching to move a cohort substantially.

4 · Code repair — Cognizant's Automata Fix

You are given a **partially written or buggy program** and must complete or fix it. Documented recurring bug types: **off-by-one errors, missing return statements, incorrect loop termination**. Candidate sources suggest a clean, optimised fix is the main signal for the higher GenC tracks.

- A binary search that loops forever on a missing element — find and fix the boundary update. *representative*
- A string-reversal function that drops the last character. *representative*
- A recursive factorial missing its return on the base case. *representative*
- A linked-list insertion that loses the head pointer. *representative*
- A sorting routine whose inner loop bound causes it to skip the final comparison. *representative*

How to teach it: take any five working programs the students wrote last semester, introduce one defect into each, and hand them back. The exercise costs nothing and is a near-exact simulation of the round.

5 · Written and spoken English production

Four companies test **production**, not comprehension, and all four eliminate on it.

- **Wipro** — a written essay of roughly 100–300 words in 20 minutes, scored on organisation, diction and ideas, with marks deducted for spelling, grammar, punctuation and relevance. One reported topic: *"Indian agriculture today versus ancient times."*
- **Tech Mahindra** — write a **narrative story based on a displayed picture**, within a character limit and time window. Marked on grammar, punctuation and logical coherence. Eliminating.
- **Accenture** — an AI-graded spoken-English assessment, **63 questions across six sections**, as Round 1, before any aptitude or coding.
- **HCLTech TechBee** — the Versant AI-graded spoken test: fluency, pronunciation, listening.
- **JPMorgan** — a **HireVue recorded video interview**: two pre-recorded behavioural questions, about a minute to think, two attempts each. Candidates who cleared the coding test have been eliminated here.

Practice set for a placement cell: ten timed 250-word essays on general topics, marked strictly on grammar and structure; five picture-prompt narratives; and — the highest-value exercise — **recorded one-minute video answers** to the standard behavioural questions, played back to the student. Most students have never heard themselves speak and are shocked by it. That shock is the lesson.

6 · Psychometric and behavioural inventories

Capgemini's **ADEPT-15** (forced choice: which of two statements describes you better), Tech Mahindra's 86 questions in 25 minutes, LTIMindtree's 50 questions in 20 minutes. These are **generally not used for elimination** but are mandatory and are profiled for the HR round. Tell students two things: answer consistently, and do not try to construct an ideal persona — forced-choice instruments are specifically designed to detect it.

D 2

Data structures and algorithms, by topic

The VTU Data Structures and DAA courses cover almost every structure in this section. What they do not do is ask students to **recognise which structure an unseen problem needs, implement it under time pressure, and state the complexity out loud**. That recognition skill is what every round in this section is actually testing.

The gap, stated precisely

A typical VTU Data Structures laboratory asks students to *implement a stack*, *implement a binary search tree*, *implement Dijkstra's algorithm*. Every one of those is a named structure with a known implementation. **Not one interview question in this entire section names the structure.** They describe a situation and the candidate must infer it. That inference is the whole skill, and it is untaught.

Arrays, prefix sums and matrices

- Count contiguous subarrays summing to *k*. **Flipkart**
- Replace each array element with the product of all the others — then do it in O(N) time, then in O(1) extra space **without division**. **Intuit**
- Maximum product subarray. **Zepto, Chargebee**
- Sort an array of 0s, 1s and 2s in one pass. **Goldman Sachs**
- Find the first and last occurrence of an element in a sorted array. **Zepto**
- Merge two sorted arrays in O(n) using the spare space in one of them, with no extra memory. **Salesforce**
- Find $i < j < k$ such that arr[i] < arr[k] < arr[j]. **Navi**
- Search a value in a row-wise and column-wise sorted matrix. **Goldman Sachs, Freshworks**
- Given a matrix of characters, count the maximum occurrences of a four-letter word moving in four directions without revisiting a cell. **Walmart**
- Rotate a matrix by 90 degrees in place; spiral traversal; set an entire row and column to zero in O(1) space. *representative*

Strings

- First non-repeating character in a single pass. **Flipkart**
- Longest substring with at most *k* distinct characters. **Groww**
- Longest valid parenthesis **subsequence** — note, discontinuous, not substring. **Zomato / Eternal**
- Given distinct binary strings of length N, construct a length-N binary string not present in the array. **Zepto**
- Count the number of distinct strings obtainable from *n* rotations of a string, in O(n). **Salesforce**
- Count numbers greater than B that can be formed by picking digits in order from a string of up to 47 digits — a digit-DP problem. **PhonePe**
- Left-align text to a fixed width. **Salesforce** — logically simple, implementation-heavy; a good test of care rather than cleverness.
- String matching: code brute force, then discuss KMP. **Sprinklr**
- Longest common subsequence; palindromic partitioning; group anagrams; minimum window substring. **Goldman Sachs, Chargebee** *and representative*

Two pointers, sliding window and binary search on the answer

Three patterns that appear constantly in assessments and almost never in a college lab, because none of them is a named data structure.

- Sliding window appears at **Flipkart, Swiggy, Groww and InMobi** in the documented accounts.
- Maximum number of visible points from an origin, given angles. **InMobi** — solved with a sliding window after hints.
- Find the maximum *K* such that repeatedly flipping K-length substrings turns a binary string to all zeros. **Uber**
- Longest even-length substring whose two halves have equal digit sums. **Lowe's**
- Minimum capacity to ship packages within D days; minimum eating speed to finish piles in H hours. *representative — the canonical "binary search on the answer" shape*
- Count triplets summing to zero. **Swiggy**

Stacks and queues — including the monotonic stack

The VTU lab implements a stack. Interviews ask you to *notice you need one*.

- Stock Span. **Rakuten, Juspay**
- Asteroid Collision. **Navi**
- Next smaller element — reduce to a stack solution, then rework it to constant space. **InMobi**
- Stack-based removal of repeated values. **Goldman Sachs**
- Design a stack that returns its minimum in **O(1) time and O(1) space**. **Arista**
- Evaluate postfix and prefix expressions using a stack. **Goldman Sachs**
- Largest rectangle in a histogram; sliding-window maximum with a deque; implement a queue using two stacks. *representative*
- Task scheduler with a cooldown period. **Swiggy**

Linked lists

- Detect a loop in a linked list; find the middle node in a single pass. **Innovaccer, Cisco**
- Remove duplicates from a sorted list, keeping only distinct values. **Groww**
- Add two numbers represented as linked lists. **Swiggy**
- Convert a binary linked list to its decimal value. **Arista**
- Merge two sorted linked lists. **Intuit**
- Delete a node without being given the head pointer; flatten a multi-level linked list. **Chargebee**
- Doubly linked lists and their real-world uses — asked conceptually. **Qualcomm**

Trees and binary search trees

- Flatten a binary tree to a linked list. **Flipkart**
- Burning Tree — time for a fire to spread from a given node. **PhonePe**
- Check whether a binary tree is a valid BST. **Qualcomm, Freshworks, Juspay**
- BST deletion when the node has two children. **Zomato / Eternal**
- Convert a BST to a Greater Sum Tree. **Zepto**
- Replace each node's value with the sum of its left subtree. **Lowe's**
- Transform a tree in place using reverse in-order traversal with **no extra space**. **Navi**
- Build a tree from its in-order and post-order traversals. **Freshworks**
- Count the number of distinct binary trees formable from 1..n — brute force, then dynamic programming. **Swiggy; a variant at Oracle**
- **B+ tree insertion complexity, the effect of changing the node order *k*, and why a sorted array might be preferable.** **Zomato / Eternal** — note this is a *theory* question of exactly the kind the DBMS and DS courses cover, asked at interview.

Heaps and top-K

- The k most frequent numbers, using a hash map plus a size-k min-heap. **Flipkart**
- Median of a stream of running integers — the two-heap pattern. **Navi; a stock-market-themed variant at Groww**
- Kth smallest element in a sorted matrix. **PhonePe**
- Minimise the sum by repeatedly combining elements, using a priority queue. **Swiggy**
- Minimum number of refuelling stops. **Flipkart**
- Choose *k* crops to maximise profit where each has a minimum capital requirement and profit increases capital — the capital/profit greedy-heap pattern. **Uber**
- Minimum number of platforms required at a railway station. **Goldman Sachs**

Tries and prefix structures

Almost never implemented in a college lab, and asked at four companies in this study.

- Design a prefix-search structure. **Goldman Sachs**
- A trie plus BFS problem in the online assessment. **Cisco**
- Tries appear explicitly in the machine-coding round topics. **CRED**
- Implement autocomplete over a dictionary with ranking by frequency. *representative — the Google-Search shape*

Graphs

The highest-value topic in this section, and the one where the gap between lab and interview is widest. The lab implements Dijkstra on a given adjacency matrix. Interviews describe a situation and expect the student to *see a graph*.

- Check whether a graph is bipartite. **Flipkart**
- Shortest path in a graph. **Swiggy**
- Minimum time for a traveller with limited energy to reach a destination, with energy available at some nodes; return −1 if impossible. **Uber**
- Shortest Path to Get All Keys — BFS over a state space, not just over positions. **PhonePe**
- Parallel Courses III — topological sort with timing. **Navi**
- Package distribution using **disjoint set union**. **Walmart**
- **Articulation points using Tarjan's algorithm.** **Walmart** — the hardest single documented question in this report.
- An optimal route minimising operations — travelling-salesman flavoured. **Meesho**
- Number of Islands, extended. **Meesho**
- A virus-containment problem over a grid. **Darwinbox**
- Grid reachability with restricted moves; stepping numbers via BFS. **Swiggy**
- Minimum operations to reach a target, modelled as BFS. **Goldman Sachs**
- A disjoint set union problem in the online assessment. **Zeta**
- Detect a cycle in a directed graph; course-schedule feasibility; clone a graph; word-ladder shortest transformation. *representative*

Dynamic programming

- Unbounded knapsack. **Swiggy**
- Decode Ways. **Groww**
- Coin change. **Chargebee**
- A Burst Balloons variant — interval DP. **Uber**
- Grid path counting. **Goldman Sachs**
- Dynamic programming on a matrix. **Intuit**
- A hard backtracking/DP problem in the assessment. **Meesho**
- **Digit DP** — counting numbers formable from digits in order. **PhonePe**
- **Bitmask DP** — choose pairs (a, b) such that each next pair's *a* is at most the previous pair's *b*, maximising the sum of *b*. **PhonePe**
- Longest common substring; longest palindromic subsequence; knapsack — the documented “hard” set for the Infosys Specialist Programmer coding test. **Infosys SP**
- Longest increasing subsequence; edit distance; matrix chain multiplication; house robber on a tree.
  representative

Greedy and backtracking

- Matchsticks to Square. **PhonePe**
- N-Queens. **Freshworks**
- Combination Sum II with a size constraint. **Darwinbox**
- Assign limited uses of a strong hammer to planks of varying strength; output the minimum hits, which planks the strong hammer broke in descending strength, and the weak hammer's hit count. **Arista**
- A scheduling problem solvable either with arrays or with heaps plus hash maps — the interviewer wanted both approaches compared. **Meesho**
- Largest number formable from an array. **Goldman Sachs**
- Next permutation. **Goldman Sachs**

Bit manipulation, number theory and mathematics

- A bitwise problem paired with a Merge Intervals variant. **Swiggy**
- Convert a number from base 2 to base 6. **Uber**
- Divisor queries at a scale of up to 10^8, requiring precomputed divisors and GCD properties. **Walmart**
- Sieve of Eratosthenes; primality testing. **Navi, Cisco**
- Sum of the ten largest primes across subarrays. **Darwinbox**
- Counting pairs whose difference is divisible by a given value. **Goldman Sachs**
- Count set bits efficiently; single number among duplicates using XOR; swap two numbers without a temporary.
  representative

Design-flavoured DSA — the bridge to Part D 4

These sit between algorithms and system design, and are asked in DSA rounds at several companies. They are the single best preparation for the machine-coding round, because they require **designing an interface** rather than writing a function.

- Implement an **LRU cache**. Salesforce, Juspay
- Design a stack with O(1) minimum. Arista
- Design a data structure for a stated access pattern — the question asked without naming the structure. BrowserStack
- **Count word occurrences in a 1 TB file with only 8 GB of RAM.** BrowserStack — this is an algorithms question that is really a systems question.
- Indent a JSON object. Juspay
- Detect a loop in a JSON object. Darwinbox
- Design a rate limiter — appears as a DSA question, an LLD question and a system-design question depending on the company. Groww, Zomato
- Implement a hash map and discuss collision strategies; design a tiny in-memory key-value store with expiry. *representative*

How to run this as a teaching programme, not a question list

Per topic: teach the pattern and its template once. Then have each student solve **three easy and five medium** problems in that pattern, and — this is the part that is usually skipped — **state the time and space complexity aloud** every single time, to another person.

Then the two things nobody practises. First, **mixed sets**: once six patterns are taught, set problems without saying which pattern applies, because recognition is the actual exam. Second, **timed unseen problems with a spoken explanation**, because every DSA interview in this report marks the explanation as much as the solution — two companies, PhonePe and Navi, make candidates write the code **on paper**.

A realistic target: around 200 problems, properly understood, with complexity stated aloud each time, spread over two years. Not 500 memorised in the final semester.

D 3

Object-oriented design for business scenarios

The VTU OOP course teaches encapsulation, inheritance, polymorphism and abstraction, and examines them as definitions. Every company in this report that asks about OOP asks something different: **here is a business. Give me the classes.**

What is actually being marked

Across Uber, Meesho, Zeta, InMobi, Razorpay and Swiggy, the documented evaluation criteria for these rounds are consistent: **choice of data structures**, **adherence to object-oriented design principles**, whether responsibilities are placed on the right class, whether the design can absorb a new requirement, and whether the candidate **asked clarifying questions before designing**. Uber's documented problem was *deliberately vague* and the successful candidate asked many questions before writing anything. That is not a detour before the real answer. It is part of the answer.

The five business scenarios documented against real companies

Food delivery — Swiggy or Zomato

Model: Customer, Restaurant, MenuItem, Cart, Order, DeliveryPartner, Payment.

What the design must show: a base `User` type with Customer and DeliveryPartner as subtypes; an **abstract `Payment`** with UPI, Card and Cash implementations; correct access modifiers; immutable order lines once an order is placed; order state transitions.

The follow-up that separates candidates: add a new payment method, or a new discount type, without modifying existing classes. If the design requires editing a switch statement, it has failed the open-closed principle and the interviewer will say so.

Ride hailing — Uber or Ola

Model: Rider, Driver, Vehicle (Auto, Mini, Sedan), Ride, and a pricing strategy.

What the design must show: ride **states** and the legal transitions between them (requested, matched, started, completed, cancelled); **polymorphic pricing** covering surge, distance and time; driver-matching logic separated from the Ride entity.

Documented live variant: Meesho gave candidates a ride-sharing system with **function names and parameters supplied** — add ride, find ride, book, cancel — one hour to fill in the logic, with one test case that had to pass. The remaining 30 minutes went to **OOP basics, SOLID principles, and OS concepts — mutexes and race conditions.** Meesho, 2025

Social media — Instagram

Model: User, Post as a base type with Photo, Video and Reel subtypes, Comment, Like, Follow.

What the design must show: an interface such as `Notifiable`; a defensible choice between **composition and inheritance** for post content; correct `equals` and `hashCode`; and avoidance of a god-class `User` that accumulates every behaviour in the system.

Multi-device controller Uber, documented

"Design an application that controls multiple devices — a fan, a TV — with four functions. There are common commands such as ON and OFF, plus device-specific commands."

The requirements were deliberately vague. The successful candidate asked many clarifying questions, began with interfaces, and then switched to a base `Device` class with inheritance. Both answers are defensible; the interviewer was watching the reasoning for the switch.

Hospital appointment system InMobi, documented

"Design an appointment system for a hospital with patient priorities." Open-ended, and **explicitly evaluated on choice of data structures and adherence to object-oriented design principles**.

A teaching set of twenty business scenarios

These are *representative* — written to match the documented pattern, for use as classwork and internal assessments. Each should be given as a **two-line business description with deliberate ambiguity**, exactly as the real rounds do, and the student should be required to ask questions before designing.

#	Scenario	The design decision it forces
1	**Movie ticket booking (BookMyShow)**	Seat locking and double-booking prevention; show, screen and theatre relationships
2	**Railway reservation with waitlist (IRCTC)**	Quota handling; waitlist promotion on cancellation; berth allocation rules
3	**Library management**	Copies versus titles; borrowing limits; fine calculation as a replaceable policy
4	**Parking lot**	Vehicle-size-to-slot mapping; multiple pricing strategies; multi-floor extension
5	**Splitwise — expense sharing**	Equal, exact and percentage splits as polymorphic types; simplifying debts
6	**Vending machine**	State pattern; coin change; inventory depletion
7	**Elevator system**	Scheduling strategy; multiple cars; direction and request queues
8	**ATM**	Transaction states; note dispensing; failure and rollback

#	Scenario	The design decision it forces
9	**Hotel booking**	Room types; availability over date ranges; cancellation policies
10	**Online bookstore with coupons**	Combinable versus exclusive discounts — the exact problem Zepto posed
11	**Food court order management**	Multiple vendors in one order; partial fulfilment; order splitting
12	**Course registration portal**	Prerequisites; seat caps; clash detection; waitlists
13	**Hostel room allocation**	Preference matching; constraints; reallocation
14	**Chess or Ludo**	Rule validation; turn management; board state. **Ludo asked at Zeta**
15	**Snake and Ladder**	Board configuration; multiple players; game termination. **reported at Swiggy**
16	**Contact management system**	Search, merge and duplicate detection. **asked at CRED**
17	**Notification service**	Channel abstraction (SMS, email, push); retry and failure policy
18	**Inventory with expiry**	Batch tracking; FIFO depletion; expiry sweeps
19	**Attendance and leave management**	Policy as a strategy; approval chains; accrual rules
20	**Cab fare calculator with surge**	Pricing as a replaceable component; time and zone rules

The theory students must be able to use, not define

These appear by name in interviews at Uber, Meesho, Zeta, Freshworks, Juspay, Target, Qualcomm, Micron and Salesforce.

- **The four pillars** — but asked as "give me a real-life example of each from your own project". **Target, Qualcomm, Intuit, Zeta**
- **SOLID**, with a before-and-after refactor. **Meesho names SOLID explicitly**
- **Abstract class versus interface** — and when you would choose each. **Salesforce**
- **Composition versus inheritance** — the single most common follow-up in these rounds.
- **Static versus dynamic binding**; runtime polymorphism; copy constructors; inner classes. **Salesforce**
- **`equals` and `hashCode`**, and what breaks if you override one but not the other.
- **Design patterns by name**, each with a business example: **Strategy** (used in the Swiggy answer for discounts), **Factory**, **Singleton** and **MVC** (both named in the Razorpay round), **Observer**, **State**, **Command**, **Adapter**, **Decorator**, **Builder**.
- **Concurrency in design** — locks, race conditions, idempotency, and preventing double booking or double payment. **Meesho asks mutexes and race conditions directly in the design round**

D 4

Machine coding

The single largest gap between what colleges teach and what product companies test. You are given a business problem and 60 to 150 minutes to build a **working application, entirely in memory** — no database, no user interface, no framework — and then defend it.

What the round actually is

Parameter	What the documented accounts say
Duration	**60–150 minutes.** Swiggy and Groww 60–90; Flipkart 90; CRED 150; Razorpay 150.
Storage	**In memory only.** "No database is required — an in-memory store such as a HashMap is enough." One CRED candidate records: *"No UI / API was required. Only needed to write the logical functions."*
Where you write it	Your own IDE, a shared Google Doc (Groww), HackerRank (CRED), or the company's own platform (Uber).
The first criterion	**The code must actually run.** Every account names this as the dominant requirement. Groww: "the code had to run." Meesho: one test case that must pass. Arista: working code passing the sample tests.
Then	All stated requirements met; **modular and readable**; follows OOP principles and uses a design pattern where appropriate; **easy to extend with new functionality**; handles the custom test inputs the interviewer supplies.
And often	A **separate review round** where interviewers read your code and ask you to justify it (Flipkart, 45 minutes; CRED, a code-evaluation discussion days later). CRED also required **unit tests within the same 150 minutes**.

Why a lab record does not prepare a student for this

A lab experiment has a known answer, an unambiguous specification, no time pressure worth the name, and no requirement that the result be extensible. A machine-coding round has an **ambiguous business specification**, a hard clock, and a reviewer who will add a requirement at the end to see whether your design survives it. The two activities share only the programming language.

Documented problems, by company

Every problem below is attributed to a real candidate account. This is the most valuable single list in the report, because these are the actual problems.

Company	Problem
Flipkart	**COVID vaccination centre system** with nine stated requirements, 90 minutes, then run it and explain it

Company	Problem
Swiggy *Dec 2025*	**Shopping cart** — add items, payment, multiple discount types. Solved with the **Strategy** pattern
Swiggy *Mar 2026*	**Driver-to-order assignment** — VIP and non-VIP orders, driver ratings, multi-condition matching. Deliverables included **data-structure choices and a dry run**
Razorpay	**An in-memory database**, 2.5 hours, demonstrated over screen-share against interviewer test cases
Razorpay	**Photo-sharing app** — six features plus one bonus, 90 minutes, discussion centred on MVC and Singleton
Groww	**Group chat system** — admins create up to 5 groups, users join up to 20, admins add and remove members. 90 minutes in a shared document
Groww	**Tiered rate limiter** for image downloads — premium 100/min, general 10/min, excess **queued rather than rejected**
Meesho	**Ride-sharing system** — add, find, book, cancel; function signatures supplied, logic to be filled in
CRED	Helper functions and structure supplied; implement the logic cleanly. **OOP and Tries**; unit tests required; 150 minutes
PhonePe *experienced*	**Activity tag logging system** — hierarchical tags on activities, counting tag usage across entities. 90 minutes
Uber	**Multi-device controller** — fan and TV, common plus device-specific commands, deliberately vague requirements
BrowserStack	**A `tail -f`-style log watcher**: a server streaming a remote log file plus a web client updating in real time
BrowserStack	**A stateless web service** that starts, stops, cleans up and reports the active URL for Chrome and Firefox, without Selenium dependencies
Zoho	**A booking system** — booking, availability checking, cancellation, chart preparation. Marked on quality, modularity, boundary cases and efficiency
Zeta	**Ludo** — design it, then code it, then **find and fix the bugs in your own code**, then discuss its limitations
Freshworks *take-home*	Build a **file-based key-value data store**
Innovaccer *take-home, stale*	Build an **entry-management web app** — check-in collects visitor and host details and notifies the host; check-out verifies and sends a summary
Rakuten	Implement a **2048-style tile puzzle** in the online assessment

Also named in practice sets tagged to Flipkart, Uber, Swiggy, Razorpay, CRED and PhonePe — but **not attributable to any single company**: parking lot, Splitwise, chess move validator, snake and ladder, 2048, tic-tac-toe, library management, Trello, an in-memory key-value store, a distributed queue.

A 20-problem practice set for a skill lab

Graded by difficulty, all solvable in memory in 60–120 minutes, all extensible so the reviewer can add a requirement at the end.

Level	Problems
Starter **60 min**	Tic-tac-toe with win detection · a to-do manager with priorities and tags · a student attendance register with reports · a simple vending machine · snake and ladder for N players
Core **90 min**	Parking lot with multiple vehicle sizes and pricing · Splitwise with equal, exact and percentage splits · a library system with copies, borrowing limits and fines · an ATM with transaction states · a movie-ticket booking system with seat locking · a shopping cart with stackable discounts
Hard **120 min**	An in-memory key-value store with expiry · a tiered rate limiter with queueing · a group chat system with membership limits · a ride-matching engine with driver ratings · an elevator controller for multiple cars · a logging library with levels, sinks and formatting · an in-memory file system with directories and search · a job scheduler with dependencies and retries

How to run this as a lab — the five rules that make it real

1. Give an ambiguous specification. Two or three lines of business description, not a numbered requirement list. Ambiguity is the point; students must learn to ask. Uber's documented problem was deliberately vague.

2. Enforce the clock absolutely. At 90 minutes, collect what exists. A half-finished program that runs beats a complete one that does not compile, and students must feel that.

3. Ban persistence. No database, no file I/O, no framework. In-memory collections only. This is not an arbitrary constraint — it is what the real round specifies, and it forces design attention onto the model rather than the plumbing.

4. Run the code in front of them. With **your** test inputs, not theirs. This single step changes student behaviour more than any amount of instruction.

5. Then add one requirement. "Now add a second discount type that stacks with the first." "Now support a second vehicle size." How much of the existing code has to change *is* the design grade. A student who has to rewrite three classes has learned, in one minute, what the open-closed principle means — which no lecture achieves.

D 5

Low-level design

Where machine coding asks you to *build* it, the low-level design round asks you to *specify* it: classes, relationships, interfaces, data structures and concurrency — usually on a shared document or a whiteboard, sometimes followed by code.

Documented LLD problems

Company	Problem and what was probed
Uber	**Design MS Excel.** Also the multi-device controller described in D 3
PhonePe	**Design Twitter** — post, follow and unfollow, user timeline, user profile — covering **both LLD and HLD in one round**
Groww	**Tiered rate limiter** with queueing — a separate 60-minute round after the machine-coding round
Zepto	**Design an e-commerce application**: database schema and component interaction, key services and API endpoints, **stock management including quantity and product expiry**, **discount coupons and whether they can be combined**, design patterns and OOP principles
Zepto	**Design a database schema for Uber**
Ola *SDE-2*	**Ola's internal notification system** — requirements, high-level design, low-level components, **skeleton code, APIs and database schemas**, 90 minutes with escalating follow-ups
Zeta	**Library management system** in a shared document, followed by pseudocode. Then **Ludo** — design, code, debug
CRED	**Contact management system**; plus a round weighted to **design trade-offs rather than code**
InMobi	**Hospital appointment system with patient priorities** — marked on data structures and OO principles
Freshworks	Class diagrams on paper; LLD a separate round at senior levels; HLD examples include **Instagram with database and caching follow-ups**, a URL shortener, a parking lot
Rakuten	"Design the **high-level backend of a browser**, including state management"
Juspay	**LRU cache** design; LLD in the role-dependent third technical round, alongside distributed systems and payment-system architecture

The canonical LLD drill set

These are the problems the industry has converged on. A student who has genuinely designed and coded eight of them can handle an unseen ninth.

- **Parking lot** — vehicle sizes, slot allocation, pricing strategies, multi-floor extension.
- **Splitwise** — split types as polymorphic classes, balance simplification.
- **BookMyShow** — seat locking, concurrent booking, payment timeout and release.
- **Elevator system** — scheduling strategy, multiple cars, request queues.
- **Vending machine** — the State pattern, coin change, inventory.
- **Chess** — move validation per piece type, board state, check detection.
- **Logging library** — levels, multiple sinks, formatters, asynchronous writes.
- **Notification service** — channel abstraction, retry policy, templating.
- **Rate limiter** — fixed window, sliding window, token bucket, and the difference between them.
- **In-memory key-value store** — expiry, eviction policy, iteration semantics.
- **Food delivery, ride hailing, e-commerce cart** — as set out in D 3.

The concurrency questions that come attached

LLD rounds at Meesho, CRED and Zeta all reached into concurrency, and this is a genuine blind spot. Students should be able to answer, about their own design: **what happens if two users book the last seat at the same time?** Where is the lock, what exactly does it protect, and for how long? What makes an operation **idempotent**, and why does a payment endpoint need to be? What is a **race condition** and where is one in your design? What is the difference between a **mutex and a semaphore**, and which does your design need? **Meesho asks mutexes and race conditions directly; CRED asks about concurrency and in-server caches**

D 6

System design, with scale and cost

Increasingly asked of freshers, and almost never taught. The Software Engineering course covers process models; it does not ask a student how many servers a system needs or what it would cost per month to run. That arithmetic is what this round is about.

Who actually asks freshers to design systems

Documented for freshers: Arista (a full 50-minute round), SAP Labs (inside Technical Round 2, named explicitly alongside cloud), Rakuten (LLD plus "design a browser backend"), Adobe (LLD and design patterns in Technical 3), Zomato/Eternal (architecture *is* the interview), Zepto, and informally at Salesforce (whiteboard your own project's system flow) and Cisco (project scalability and deployment, inside the managerial round).

Not reported for freshers at: Flipkart, Target, Walmart's intern track, JPMorgan, Goldman Sachs, Qualcomm, Oracle, Micron, Lowe's, or any of the IT services firms.

The pattern is sector-linked: **infrastructure, networking and enterprise-software employers design-test freshers; consumer-internet companies mostly use machine coding instead.**

Documented system-design questions

Arista — 50 minutes, on-campus, fresher

Design a **URL shortener** that creates short links, **deletes them after a set period**, and **handles concurrent reads and writes in a multiprocessor environment**. The discussion spanned operating systems, data structures and algorithms — and ended with *"how would you implement a C++ STL vector in C?"*

Zomato / Eternal — the whole interview, December 2025

Design **online ordering**: user flow, restaurant-manager flow, **API payloads**, backend steps, **database schema**. Then SQL versus NoSQL trade-offs; **geo queries for nearby restaurants**; many-to-many manager-to-restaurant mapping via a junction table; service and controller layering; and image uploads — where the candidate proposed base64 and **the interviewer expected pre-signed URLs**.

Round two added: microservices versus monolith; **message brokers — RabbitMQ, Redis Streams, Kafka**; **rate limiter design with a sliding window**; real-time support chat; **MongoDB indexing, compound indexes and the left-prefix rule**; and handling external outages with fallbacks.

Also documented: "Design Twitter" covering LLD and HLD together (PhonePe); "build Google Sheets" (Swiggy, 2021); Ola's internal notification system with skeleton code, APIs and schemas; Instagram with database and caching follow-ups (Freshworks); the high-level backend of a browser including state management (Rakuten); and — at BrowserStack, as a DSA question that is really a systems question — **count word occurrences in a 1 TB file with 8 GB of RAM**.

The building blocks a student must be able to name and justify

- Load balancers; stateless application servers; horizontal versus vertical scaling.

- Caching — where to put it, what to evict, and what happens when it is cold.
- SQL versus NoSQL, and the honest answer that it depends on the access pattern; replication; **sharding** and the choice of shard key.
- **Indexing** — including compound indexes and the left-prefix rule, and **dense versus sparse** indexing.
- Message queues and brokers; synchronous versus asynchronous processing; retries and dead-letter handling.
- Object storage and CDNs — and **pre-signed URLs** for uploads, which is the specific thing the Zomato candidate got wrong.
- Search indexes; geo-indexing for location queries.
- **ACID properties and the CAP theorem** — asked directly at Zepto.
- Rate limiting: fixed window, sliding window, token bucket.

The part almost nobody teaches: scale and cost arithmetic

A student should be able to do this on paper in five minutes, for any system they are asked to design.

Step	Quantity	How to get it
1	**Daily and monthly active users**	State an assumption out loud and proceed. Interviewers want the assumption named, not guessed silently.
2	**Requests per second, average and peak**	Actions per user per day × DAU ÷ 86,400. Then a peak multiple — typically 3× to 10×.
3	**Read-to-write ratio**	Almost always read-heavy for consumer products. It decides whether you cache or shard first.
4	**Data per day and per year**	Bytes per record × records per day. Then multiply by the retention period.
5	**Bandwidth**	Response size × requests per second. This is where image and video systems become expensive.
6	**Monthly cost**	Map the above onto published cloud prices — compute, storage, egress. **Egress is the line students always forget.**

A teaching set of twelve, each with scale and cost

Twitter or X timeline · Instagram feed · **IRCTC Tatkal booking** · WhatsApp messaging · a URL shortener with expiry · a food-delivery order and tracking system · a ride-matching service · a notification service across SMS, email and push · a video-streaming service · an online-exam platform for 50,000 concurrent students · a campus attendance system across 60 colleges · a rate-limited public API.

Why the cost column matters more than it appears to

Two designs can both “work”. Only one of them is affordable, and the difference between them is often a factor of ten. Asking a student to produce a monthly cost estimate forces every vague decision — store the images where? keep the logs how long? cache what? — into a number they must defend. It is the fastest way to convert hand-waving into engineering, it is genuinely enjoyable to teach, and it is the single clearest marker of a student who has thought about production rather than about marks.

It also happens to be exactly what a forward-deployed or product engineer does in their first year on the job.

D 7

The project cross-examination

The only round that appears in all four employer segments, and the one students most consistently misjudge. They prepare a presentation. They are given an interrogation.

The finding, stated once, plainly

Across Flipkart, Swiggy, Zomato, Meesho, Razorpay, CRED, Zepto, Navi, HackerRank, Darwinbox, Juspay and Chargebee, this round is an **engineering-judgement cross-examination of the student's own work**. At HackerRank it is the *decisive* round — the candidate shared their screen and walked through open-source pull requests from a previous internship, answering questions about that code.

The practical consequence: **one project understood completely beats five that can only be described.** A student with three shallow resume entries is in a worse position than a student with one real system they built, broke and fixed.

The documented questions

On the decisions you made

- Why this approach, and why this technology stack? **Meesho, Razorpay, Zepto, Qualcomm**
- Why MongoDB, and what were the alternatives? **Qualcomm**
- Why React, how do components work, and what did you check before deploying the login page? **Intuit**
- Walk me through the architecture of each component and the role it plays. **Razorpay — the candidate's own stated advice**
- Draw the system flow of your project on the whiteboard. **Salesforce**

On whether it would survive contact with reality

- Would this work in production, and how would you improve it? **Meesho**
- How did you make it scalable? **Zepto**
- Redesign your own backend and tell me what improved. **Zomato / Eternal**
- Tell me about a **production incident you handled**. **Swiggy, Dec 2025**
- Tell me about a project that **failed**, or looked like it was failing. **BrowserStack**
- How would you log this for monitoring in production? **BrowserStack**
- What happens when an external dependency goes down? **Zomato / Eternal**

The one that catches almost everyone

- **You claim a "15% cost reduction" on your resume. How exactly did you calculate that number?** **Zepto**

Every quantified claim on a resume is an invitation to this question. A student who cannot reconstruct the measurement should remove the number. A student who can has just demonstrated engineering rigour in thirty seconds.

On the surrounding knowledge the project drags in

- SQL versus NoSQL for your use case; transactions; **ACID and the CAP theorem**. **Zepto, Navi, Eternal**
- Indexing in MongoDB versus MySQL and its drawbacks; **compound indexes and the left-prefix rule**. **Zepto, Eternal**
- Microservices versus monolith — and which did you have, really? **Eternal**
- JWT authentication, role-based routing, API design in your own project. **Zepto, Oracle**
- Spring Boot annotations, dependency injection, the JVM and garbage collection — if Java appears anywhere on the resume. **Swiggy, Oracle, Rakuten**
- Linux boot process and shell configuration; Docker and Kubernetes basics; Git workflow. **Oracle, Darwinbox**

What a college should change about final-year projects

Require a design document — not a report. Architecture, data model, API contract, the alternatives considered and the reason for each decision.

Require it to be deployed and running, with a public URL and a public repository. “Evidence beats credentials” is the most-repeated finding in this report, and a deployed project is the evidence.

Require a measurement. One number — latency, throughput, cost, error rate — measured before and after a change the student made, with the method written down.

Run an adversarial viva. Not “explain your project”, but “why did you choose this; what breaks at ten times the load; what would you do differently; prove that number.” Thirty minutes of this in the seventh semester is worth more than any amount of mock aptitude testing.

Prefer one deep project to three shallow ones, and say so in the assessment rubric, because students optimise for the rubric.

D 8

Computer science fundamentals

This is the section where the VTU syllabus already contains the answer. Operating Systems, DBMS, Computer Networks and OOP are all core courses. The gap is not coverage — it is that students learn these for an examination in semester four and cannot use them in conversation in semester seven.

> ### The official statement worth putting in front of every student
>
> Salesforce, on its own engineering blog, names what it screens for: **"data structures and algorithms, object-oriented programming, operating systems, DBMS, and computer networks"** — and recommends **college course notes** as preparation for the fundamentals. There is no clearer endorsement available that the standard syllabus is the right syllabus. The problem is retention and activation, not content.

Programming, memory, state and types

Asked most heavily at Qualcomm, Arista, Micron, Salesforce, Freshworks, Zoho and in every code-output section.

- What is a variable, in memory? Stack versus heap allocation — what lives where, and what happens at function exit. *representative; the foundation for everything below*
- Pass by value versus pass by reference; what a pointer actually holds; pointer arithmetic. **Freshworks asks C pointers directly**
- **Code-output prediction** — a standard section at Walmart, Qualcomm, Micron, HCLTech, Darwinbox and Zoho.
- **Number-system conversion, binary to hexadecimal.** **Qualcomm**
- Static versus dynamic binding; type conversion; copy constructors; inner classes. **Salesforce**
- Time complexity of ordered versus unordered maps in C++. **Intuit**
- **JDK versus JRE versus JVM; garbage collection; Collections; multithreading.** **Swiggy — asked verbally for 25 minutes**
- Java internals and exception handling. **Rakuten**
- **"How would you implement a C++ STL vector in C?"** **Arista** — the best single question in this report for testing whether a student understands what a data structure *is*.
- Memory leaks: what causes one, how would you detect it, and what does a garbage collector not save you from? *representative*

Operating systems

The most heavily examined fundamentals subject in this study. Documented at Cisco, Micron, Intuit, Qualcomm, Walmart, Arista, Lowe's, Oracle, Meesho, Freshworks and Zoho.

- **Process versus thread** — what is shared and what is not. **Cisco**
- **Process states** and the transitions between them. **Intuit**
- **CPU scheduling — FCFS, SJF, Round Robin** — and why scheduling matters at all. **Intuit, HCLTech**
- **Deadlock** — the four conditions, detection, prevention, avoidance. **Cisco, HCLTech**
- **Starvation** and the **critical section** problem. **Cisco**
- **Semaphores versus mutexes** — and which your design needs. **Cisco, Micron, Meesho, Salesforce, Navi**
- The **producer–consumer** problem. **Salesforce**
- **Race conditions** — where one is in your own design. **Meesho**
- **Paging, page-replacement algorithms and thrashing.** **Micron, Intuit, Freshworks**
- Virtual memory; segmentation versus paging; internal versus external fragmentation. *representative*
- **Zombie processes**; orphan processes. **Micron**
- **Inter-process communication** — pipes, shared memory, message queues, sockets — and when you would choose each. *representative; implied across the OS questioning at Cisco, Micron and Arista*
- **Real-time operating systems** — what makes one real-time. **Cisco**
- **RAID** levels. **Micron**
- Linux: the **boot process**, shell configuration, basic command-line proficiency. **Oracle, Darwinbox**

Database management systems

Oracle's centre of gravity, and heavily examined at Micron, Intuit, Target, SAP Labs, Rakuten, Zepto, Freshworks and Zeta.

- **Normalization** to third normal form, with a real table. **Intuit, Micron, Oracle, Zeta, Sprinklr**
- **ACID properties.** **Micron, Zepto** — and the **CAP theorem** alongside it. **Zepto**
- **Keys** — primary, foreign, candidate, composite — and table relationships. **Micron, Oracle, Target**
- **DDL, DML and TCL commands.** **Target, June 2026**
- **Transactions and transaction management.** **Target** — and **database locking**. **Salesforce, Rakuten**
- **Joins**, including a **self-join**, and a **COUNT over a UNION of joins**. **Oracle**
- **Indexing — dense versus sparse**; compound indexes and the **left-prefix rule**; the cost of an index on writes. **Oracle, Zepto, Eternal**
- **The Nth-highest-salary query**, with and without a nested query. **Micron, Innovaccer** — the most-asked SQL question in India.
- **SQL versus NoSQL** and how each scales. **Oracle, Zepto, Rakuten, Navi, Eternal**
- **GROUP BY with HAVING**; aggregate functions; subqueries versus joins. **HCLTech**
- **B+ tree insertion complexity** and the effect of node order. **Zomato / Eternal**

Computer networks

Weighted uniquely high at Arista and Cisco; also at Target, Qualcomm, Navi, Zeta, BrowserStack and Juspay.

- **The seven layers of the OSI model.** Target, June 2026 — asked exactly like that.
- **TCP versus UDP** — the guarantees each gives and which you would choose for video, for payments, for a game. Zeta
- The **TCP three-way handshake**; flow control; congestion control. *representative*
- **What happens when you type a URL into a browser and press enter?** Zeta — the most reliable single network question in Indian interviewing; it touches DNS, TCP, TLS, HTTP, rendering and caching in one answer.
- **DNS**, explained in detail. Cisco — asked in the managerial round
- **HTTP versus HTTPS**; what TLS actually adds. BrowserStack, Zeta
- **HTTP status codes** — and which you would return for each failure in your own API. BrowserStack
- HTTP methods and idempotency; headers; the **User-Agent header**. BrowserStack
- **Sockets** — and the relationship between a socket, a port and a process. Zeta
- **IP address types**; public versus private; NAT. Cisco
- **DHCP**; network topologies; **FTP active versus passive mode**. Cisco
- **Routing, switching, TCP/IP and SDN.** Arista
- Encryption basics and cryptography. Salesforce, Navi

The teaching problem, and the cheapest fix in this report

None of this is missing from the VTU syllabus. All of it is examined in semesters three to five and then left to decay. By the time a student sits in front of a Cisco or Target interviewer in semester seven, two years have passed since they last thought about paging or the OSI model.

The fix is not more content. It is **a fundamentals revision strand running through the final year** — one hour a week, oral, in pairs, using exactly the questions above. It requires no new syllabus, no new faculty expertise and no budget. It is, measured against the effort, almost certainly the highest-return intervention available to any department reading this report.

D 9

Communication and behavioural

Treated as the soft round and prepared for accordingly. It is not soft: at five of seven IT services firms it eliminates candidates before any technical evaluation, and at product companies the hiring-manager round is frequently the last gate before an offer.

The questions that recur across every segment

Opening and motivation

- Tell me about yourself. *HCLTech's documented advice: under 90 seconds, ending with your interest in the company*
- Why this company, specifically? **asked at essentially every company in this study**
- What do you know about us? **Oracle asks about its products; Lowe's expects you to have read the company website and social channels**
- Where do you see yourself in three to five years? Any plans for higher study?
- Are you willing to relocate, and to which locations? **Cisco asks Bangalore, Chennai or Pune; LTIMindtree lists five cities**

The behavioural set — prepare eight stories

- A difficulty in your last project and how you solved it. **JPMorgan HireVue**
- A time you disagreed with a manager or a colleague. **Uber, BrowserStack, Razorpay, Zepto**
- A teammate who was not pulling their weight — what did you do? **Uber**
- Convincing a team lead about a technical trade-off. **Razorpay**
- Describe a failure. **Razorpay; BrowserStack asks for a failed project**
- Critical feedback you have received. **Zepto**
- A time you took ownership of something nobody asked you to own.
- A time you had to learn something quickly — and how you learn in general. **Flipkart asks how you stay current; Zepto asks about your learning approach**

Scenario questions — the ones with no right answer

- “Delivery times have spiked in one city. What would you investigate?” **Swiggy**
- “If you had joined in 2018, how would you have designed the digital lending section of the app?” **Navi**
- “What would you do if a production deployment failed at 2 a.m. and you were on call?” *representative*

These are marked on **reasoning structure**, not on the conclusion. The expected shape is: clarify the question, state assumptions, break the problem into parts, work through them aloud, and say what you would check first and why.

Eight STAR stories — the single most portable preparation a student can do

Situation, Task, Action, Result. Eight stories, each 90 seconds, written out and rehearsed aloud, covering: **ownership, conflict, failure, leadership, learning something fast, a technical trade-off you argued for, working with someone difficult, and a time you were wrong.**

Those eight will answer, with light adaptation, essentially every behavioural question in this report. The work is in writing them honestly and practising them out loud — not in memorising a script, which interviewers detect immediately.

Production of English — the gated formats

Recapping from D 1, because it belongs here too: **Accenture** runs AI-graded spoken English as Round 1, 63 questions across six sections. **Wipro** sets a scored written essay and a voice assessment. **Tech Mahindra** sets an eliminating picture-based story. **HCLTech TechBee** runs the Versant spoken test. **Cognizant** runs a standalone communication assessment first of three. **JPMorgan** runs a recorded video interview that has eliminated candidates who cleared the coding test.

What these measure is **fluency under time pressure** — speaking without long pauses, structuring a spoken answer, writing grammatical prose at speed. None of it is tested by a written English examination, and almost none of it is practised in a regular classroom.

A practice protocol that costs nothing

Weekly, in pairs, from the fifth semester: one student answers a behavioural question for 90 seconds while the other times and records on a phone. They swap. Then both watch their own recording. That is the entire method.

Students are startled by their own filler words, pace and posture, and they self-correct faster from one recording than from ten corrections by a trainer. Add one timed 250-word written piece a fortnight, marked strictly on grammar and structure, and a department has addressed round nine properly for the price of a WhatsApp group and a spreadsheet.

PART E

What must change

The gap between the syllabus and the nine rounds, and what each of the four audiences for this report can do about it — starting this academic year.

- E 1 The gap, mapped course by course
- E 2 What the college must do
- E 3 Measure it: periodic assessment
- E 4 What the placement cell must do
- E 5 What the student must do
- E 6 A closing argument

E 1

The gap, mapped course by course

This is the most important table in the report. Read down the right-hand column: in almost every row, the syllabus already contains the knowledge. What is missing is the **use** of it.

VTU course	What it teaches	What the interview asks that the course does not
Programming in C / Python	Syntax, control flow, a prescribed list of lab programs	Solve an **unseen** problem; **read** unfamiliar code and predict its output; **fix** a buggy program. Capgemini and Cognizant test only these last two.
Data Structures & Applications	Implement stacks, queues, lists, trees, graphs	**Recognise which structure an unnamed problem needs**, implement it in 20 minutes, and state the complexity aloud. Tries, disjoint set union and monotonic stacks appear in interviews and rarely in the lab.
Design & Analysis of Algorithms	Prove and analyse named algorithms	Pattern recognition under time pressure: sliding window, two pointers, binary search on the answer, digit and bitmask DP. None is a named algorithm.
Object-Oriented Programming	The four pillars, inheritance, polymorphism, Java syntax	**Model a real business in classes.** SOLID with a refactor. Design patterns by name with a business example. Composition versus inheritance, defended.
DBMS	The relational model, normalization, SQL, transactions	The *same topics* — normalization, ACID, joins, indexing, the Nth-highest-salary query — asked conversationally two years later, plus **installing and operating a real server** and choosing between SQL and NoSQL for a stated workload.
Operating Systems	Processes, threads, scheduling, memory, deadlock	The *same topics*, asked at Cisco, Micron, Intuit and Qualcomm — plus the ability to point at a **race condition in your own design** and say where the lock goes.
Computer Networks	OSI and TCP/IP layers, protocols	The *same topics* — and then **writing software on top of them**: sockets, HTTP methods and status codes, what happens when you type a URL, pre-signed uploads, rate limiting.
Software Engineering & Project Management	Lifecycle models, UML, process	**Capacity and cost arithmetic.** A real SRS and design document. A design review you must survive. Agile as practised, not as defined.
Web / Full Stack electives	One framework, one assignment	Three tiers across machines, deployment, CI/CD, and a **live URL a recruiter can open**.

VTU course	What it teaches	What the interview asks that the course does not
AI / ML / GenAI courses	What a transformer is; how to prompt	Build an agent; ship with an AI coding tool; **read, defend and debug code you did not write**. AI operations is 51% of AI fresher roles.
Mini and major projects	A report and a viva	An adversarial cross-examination: why this stack, would it survive production, prove the number on your resume.
— no course —	—	**Machine coding.** Build a working application in memory in 90 minutes and defend it. The largest single gap in this report.
— no course —	—	**Spoken and written English production under time pressure.** Eliminating at five of seven IT services firms.

The conclusion this table forces

Only **two rows** describe genuinely absent content: machine coding, and graded English production. Every other row describes a subject VTU already teaches, examined in a way the course does not practise.

That is good news, and it should be said clearly to any governing council: **this is not a curriculum replacement problem.** It is an application problem. The theory is already there and it is sound — Salesforce's own recruiting page recommends college course notes. What is missing is a structured place in which students *use* that theory on unfamiliar problems, under time pressure, in front of someone who asks why.

E 2

What the college must do

Four changes, in order of return on effort. The first costs almost nothing. The fourth is the structural one.

1 · Keep the fundamentals alive into the final year

The problem: Operating Systems, DBMS and Computer Networks are examined in semesters three to five and are cold by semester seven, which is exactly when Cisco, Target, Micron, Oracle, Intuit and Salesforce examine them conversationally.

The intervention: a one-hour weekly oral revision strand through the final year, in pairs, using the question bank in D 8 verbatim. No new syllabus, no new faculty expertise, no budget. Measured against the effort, this is the highest-return action in this report.

2 · Change what a final-year project has to produce

Four additions to the project rubric, none of which requires new teaching:

- A **design document** — architecture, data model, API contract, alternatives considered and the reason for each decision — not a report.
- **Deployed and running**, with a public URL and a public repository.
- **One measured number**, before and after a change the student made, with the method written down.
- An **adversarial viva**: why this, what breaks at ten times the load, prove that number.

Six in ten fresher postings now prefer or require prior work experience. A deployed, defensible project is the closest a college can come to manufacturing that evidence for every student rather than the lucky few.

3 · Teach the three rounds that have no course

Machine coding — run it as described in D 4: ambiguous specification, hard clock, in-memory only, code run in front of the student with the instructor's test inputs, then one new requirement added at the end.

Pseudocode tracing and code repair — six hours of teaching, using students' own programs from the previous semester with defects introduced. This is also the closest available proxy for the AI-era skill of judging code you did not write.

English production — weekly recorded 90-second answers played back to the student, and a fortnightly timed 250-word written piece marked strictly.

4 · Attach a structured skill lab to each semester

The first three changes are repairs. This is the structural answer, and VTU has already created the opening for it: **circular 3216 mandates five offline skill labs before the seventh semester**, explicitly beyond the prescribed curriculum, with the content left to the college.

The model Algorithms365 recommends, and has set out in detail in a companion document, is **six flagship labs of 50 hours each — 300 hours in all — one attached to each semester where VTU teaches the matching theory**:

Sem	Lab	The rounds it closes
1–2	**Foundations: logic, programming and working apps**	Round 1 — and the habit of building something that runs
2–3	**Data structures and algorithms, applied**	Rounds 2 and 3
3–4	**OOP and the data layer**	Rounds 4 and 8 — business modelling, and real databases
4–5	**Full-stack engineering: APIs, front end, deployment**	Round 7 — the deployed project that is now a screening requirement
5	**Design and practice: LLD, system design, working in industry**	Rounds 5, 6 and 9
6	**AI-accelerated engineering: GenAI, agents and quality**	The AI premium — and the judgement skill the Stack Overflow data identifies

A seventh lab in competitive programming, and an optional forward-deployed engineering capstone, run in the final year for students targeting the top of the market.

On the credit arithmetic, stated honestly

VTU credits a skill lab at 30 hours for one credit, and mandates five. A 50-hour lab therefore carries the credit in its first 30 hours, with the remaining 20 as project depth; any five of the six satisfy the minimum and the rest run as value-add. Whether a student may earn credit for more than five labs is a question worth putting to VTU directly — it changes the economics, though not the design.

And one thing to do that is not about teaching at all

Over **90% of leading GCCs work with universities**, and Karnataka's state GCC policy has an AI Skilling Council and a ₹100 crore innovation fund explicitly intended to support GCC–academia collaboration. Karnataka holds roughly 30% of India's GCCs and 32% of their talent.

A college in this state that has not approached a single GCC about curriculum input, guest instruction, problem statements for student projects, or an apprenticeship pathway is leaving the most available advantage in the country untouched. The employers are, by their own statements, looking for the conversation.

E 3

Measure it: periodic assessment

Everything in this report is advice until something measures whether it worked. The recommendation in this section is the one that makes all the others accountable — to the college, to the student, and to whichever vendor the college is paying.

The problem it solves

Most institutions already spend money on placement preparation. Aptitude training is bought in, a coding platform is subscribed to, a soft-skills vendor runs sessions in the sixth semester. At the end of the cycle the only number anyone has is the placement percentage — which arrives **eighteen months too late to act on**, is affected by a dozen things outside the college's control, and says nothing about which intervention worked.

The result is that colleges make a substantial annual investment in skilling with **no instrument capable of telling them whether it is working**. If a vendor's cohort scores no better in January than it did in August, that is the vendor's result — but without measurement, nobody can see it, and the contract renews anyway.

> ### The recommendation, in one sentence
>
> Run **periodic, proctored, interview-realistic assessments on every student from the first semester to the seventh**, keep the longitudinal record, and use it for three things: to tell students where they actually stand, to hold every training investment to a measurable result, and to give recruiters a pre-filtered shortlist instead of a crowd.

What to assess, and when

The critical design decision is this: **assess all nine rounds from Part C, not just the two that are easy to score.** An assessment programme that measures only multiple-choice aptitude and algorithm problems will produce rising scores while the real gaps — machine coding, design, project defence, spoken English — stay exactly where they were. That is worse than no measurement, because it manufactures false confidence.

Sem	What to assess	Instrument
1–2	**Logic, basic programming, code tracing**	Platform-based coding tests plus **pseudocode output prediction** — the Capgemini-style format no college tests
3	**DSA foundations; code repair**	Timed coding tests; a **debug-a-broken-program** set, Cognizant Automata Fix style
4	**DSA at interview difficulty; SQL; OS and DBMS fundamentals**	Mixed assessment — **CS-fundamentals MCQs plus coding**, which is the GCC signature format
5	**Class design from a business brief; a timed machine-coding build**	Human or AI-assisted review against the Part D 4 criteria: does it run, is it modular, can it absorb a new requirement

Sem	What to assess	Instrument
6	**LLD, basic system design, project defence, spoken English**	**Interview simulation** — increasingly AI-conducted and recorded; a 90-second recorded spoken answer
7	**Full mock loop: assessment → DSA → build → project cross-examination → HR**	End-to-end simulation of a real drive, with a verdict per round

The tools

The practice and assessment layer is mature and inexpensive. **LeetCode, NeetCode, HackerRank, HackerEarth, Codeforces, CodeChef, AtCoder, InterviewBit, GeeksforGeeks and takeuforward** cover the DSA and coding assessment need, and several are the platforms on which the real assessments in this report were actually administered — HackerRank, HackerEarth, DoSelect, CodeSignal and CoderPad all appear by name in documented candidate accounts.

The newer layer is **AI-conducted interview simulation**: tools that run a spoken or written technical interview, probe follow-up questions, and return a structured assessment. This category is moving quickly, pricing is falling, and most vendors now offer an institutional tier well below the enterprise product. For a college the realistic position is that **a lower-cost tier run frequently on every student beats a premium tier run once on a shortlist**, because the value is in the longitudinal trend, not in the precision of any single score.

One honest note: this tooling category is new enough that a college should **pilot with one cohort before committing a budget**, and should judge a tool by whether its output changes what a student does next — not by the sophistication of its dashboard.

Proctoring is no longer optional, and this is the uncomfortable part

An unproctored remote assessment in 2026 measures almost nothing. Any student can put the problem into a generative AI tool and return a correct, well-structured answer in seconds. If a college's readiness data is collected without supervision, **the data is not merely inaccurate — it is actively misleading**, and it will produce exactly the false confidence described above.

The practical response is not to ban the tools, which is neither enforceable nor desirable — Part A shows that using AI well is now a hiring criterion, and at least one employer in this study says it actively wants candidates who do. The response is to **separate the two kinds of assessment and be explicit about which is which**:

Supervised, AI-free: the readiness measurement. In a lab, on college machines, invigilated, with the record used for the shortlist. This is the number that goes to recruiters.

Open, AI-allowed: the learning work. Build with the agent, then **defend every line of what it produced** in a viva. That is precisely the PRAISE model and precisely what Part D 8 shows employers now probe.

A student who can pass the first and defend the second is genuinely ready. A student who can only do the second has learned to operate a tool, and will be found out in the first supervised round they face.

Holding the training spend accountable

Once a longitudinal record exists, a question becomes answerable that currently is not: **did the money change anything?**

- Baseline the cohort **before** any intervention, on the same instrument that will be used afterwards.

- Re-assess at a fixed interval — a semester is usually right — on an equivalent but not identical paper.
- Report the **distribution, not the average**. A training programme that lifts the top quartile and leaves the bottom half untouched is a very different purchase from one that moves the median, and the average hides both.
- Write the expected movement into the contract before it is signed.

This applies to Algorithms365 as much as to anyone else. **A skilling partner that cannot show measured improvement on an instrument the college controls is not delivering what the college is paying for**, and no institution should renew such a contract on the strength of attendance registers and feedback forms.

The shortlist — what recruiters actually want

The second use of the record is commercial, and it is the one that changes a college's relationship with recruiters.

Consider what the funnels in Part B look like from the employer's side. Walmart's Sparkplug drew roughly **20,000 registrants and selected 100**. Flipkart's GRiD drew **160,959 registrations**. A Meesho open-campus drive drew **over 1.5 lakh applications and converted 12 interns**. One Oracle campus drive tested 200+ students, shortlisted 20 and hired 5. Recruiters are not short of candidates. They are spending enormous effort filtering — and Taggd's research found **talent quality mismatch is the single biggest constraint GCCs report**, with **48% of critical roles taking 60 or more days to fill**.

The proposition a college can make, once it has the data

"Here are **fifteen** students for your five openings. Each has a two-year assessment record on supervised instruments: DSA at interview difficulty, a timed machine-coding build reviewed against your criteria, a design round, a project they can defend, and a recorded spoken assessment. Here is each student's score trajectory and here is the training they completed. Interview them, and we will send the next fifteen if you want more."

That is a materially better offer than presenting 300 students and asking a recruiter to find the five. It respects the recruiter's time, it is verifiable, and — critically — it is something a college can build **without any employer's cooperation**, using only its own students and its own assessment record.

Two cautions, stated plainly. First, a shortlist is only as credible as the integrity of the assessment behind it; one cohort of inflated scores will end the arrangement permanently, which is why the proctoring question above is not a detail. Second, **a college must not let the shortlist become a caste system**. The record exists to tell every student where they stand and what to do next — the bottom quartile needs it more than the top does. If it is used only to identify the fifteen and the rest are abandoned, the institution has bought an expensive sorting machine and taught nobody.

E 4

What the placement cell must do

Six actions, each of which can begin this season and none of which requires a budget approval.

1 · Build a drive-notification archive

Your own brochures and drive notifications are **better evidence than anything published online**, including this report — the aggregator sites are largely paraphrasing documents your office already receives. Archive every one: dated, by company, by year, with eligibility, rounds, cut-offs and outcomes. Within three cycles you will hold the most accurate picture of fresher hiring in your region, and you will be able to correct this report.

2 · Brief students on the eligibility traps before they apply

More students are lost to administrative disqualification than to technical failure, and almost all of it is preventable with one briefing. The traps are listed in full in Appendix 1; the headline ones: **60% at every level with no rounding** (Cognizant explicitly — 59.9% does not qualify); **no pending backlogs** at most companies, though Wipro allows one; **extended education disqualifies** at TCS; **education gap limits** vary from one year to three; and Infosys computes academics as a **simple average across every subject including electives and languages**, which is not your university's CGPA.

3 · Tell students how the higher tracks are actually reached

This single briefing is worth lakhs of rupees to the students who act on it:

- **Attempt every optional and advanced section.** TCS states officially that the Advanced section is mandatory for Prime or Digital consideration. The gap between skipping it and attempting it is ₹3.4 lakh versus ₹9 lakh.
- **The Infosys ₹10–21 lakh roles require a separate application.** The Specialist Programmer and Digital Specialist Engineer drive is a distinct process with its own coding test; Power Programmer runs through HackWithInfy and InfyTQ. A student sitting only the campus aptitude test can never reach them.
- **Wipro Turbo is institute-gated.** If your college is not on the list, say so rather than letting students assume performance will upgrade them.
- **Any specific cut-off number found online is unverified.** No company in this study publishes one.

4 · Open the off-campus and pool-campus routes deliberately

Most high-quality programmes do not publish their campus lists, which in practice means tier-1 first. Walmart's Sparkplug is officially restricted to 35 selected colleges. But several genuinely open doors exist and they require **individual student applications on published deadlines**, not a placement-cell invitation:

- **Cisco — Code with Cisco:** open registration, all branches, all degree programmes. Registration from 1 June in the 2026 cycle.
- **Flipkart — GRiD:** officially open to all specialisations, branches and CGPAs; 160,959 registered in its seventh edition.
- **Juspay Hiring Challenge:** all branches, no stated tier or CGPA filter.
- **Zoho:** applies through its own careers site; hires explicitly from tier-2 and tier-3 colleges.
- **Meesho Open Campus, Myntra RampUp, Atlan, Rakuten pool campus, Target's apprenticeship.**

Run a calendar. One shared document with every open programme, its registration window and its link, reviewed monthly from the third year. Most of these close before a placement cell would normally start thinking about a batch.

5 · Change what mock interviews rehearse

Mock interviews that test aptitude and ask “tell me about yourself” rehearse two of nine rounds. Add: a **timed machine-coding build** with the code run in front of the student; an **adversarial project cross-examination** using D 7; an **oral fundamentals round** using D 8; and a **recorded 90-second video answer** the student watches back.

6 · Make internships a first-year-onwards policy, not a final-year scramble

Nearly six in ten fresher postings now prefer or require prior internship experience. That single statistic should reshape the placement calendar. Internships, industry projects, open-source contributions and supervised builds all count as evidence — Atlan names **public coding contributions** as a positive signal, and BrowserStack's HR round asks directly about GitHub use.

E 5

What the student must do

A semester-by-semester plan. It assumes nothing except that you start when you read this, in whichever semester you are in.

Sem	Focus	Specifics
1–2	**Logic, and the habit of finishing**	Learn to write the logic before touching the keyboard — flowchart, dry run, edge cases. Build **three complete programs a person could actually use**, with data saved to a file. Start Git in your first month and never stop. Do not start algorithm practice yet.
3	**Data structures, by pattern**	For each pattern: learn the template, solve **three easy and five medium**, and **state the complexity aloud to another person every single time**. Arrays, strings, hashing, two pointers, sliding window, stacks, linked lists. Begin a GitHub profile with real commits.
4	**Trees, graphs, DP — and your first real database**	Finish the DSA patterns. Separately: **install PostgreSQL or MySQL yourself**, design a schema for something real, write the queries, break it. Model one business in classes — use the list in D 3.
5	**Build and deploy**	Build **one full-stack application**: an API, a front end, a database, deployed to a public URL. This is the project you will be cross-examined on for the next two years, so build something you care about. Start the weekly recorded 90-second speaking practice.
6	**Design, and AI as a tool you own**	Work through the LLD drill set in D 5 — design and **code** eight of them. Practise machine coding against a clock. Build something with an AI agent and then be able to **explain every line of what it wrote**. Write your eight STAR stories.
7	**Activate the fundamentals, and apply widely**	Oral revision of OS, DBMS, CN and OOP from D 8, in pairs, weekly. Timed unseen DSA problems, spoken aloud. Apply to **every open off-campus programme** — they close early. Rehearse the adversarial project viva until it is boring.
8	**Internship as evidence**	Treat the internship as the thing your first job interview will be about. Own one piece of work end to end, measure it, and be able to defend every decision in it.

Five things worth more than the next hundred algorithm problems

1. One deployed project you understand completely. A public URL and a public repository. Every segment in this report cross-examines it, and six in ten postings now want prior experience.

2. The ability to explain while you solve. Every DSA interview here marks the explanation as much as the answer, and two companies make you write the code on paper. Practise out loud, to a person.

3. One business modelled in classes, properly. Food delivery, ride hailing, ticket booking — then have someone add a requirement and see how much you have to change.

4. Your core subjects, still alive. Operating systems, DBMS, networks and OOP are examined conversationally at the GCCs that are now India's largest fresher employers. You already studied all of it; do not let it go cold.

5. Ninety seconds of clear speech. Recorded, watched back, improved. It gates five of the seven largest IT employers in this report before anyone looks at your code.

And one honest word about the market you are entering

It is harder than the one your seniors entered, and nobody should pretend otherwise. Entry-level IT roles have shrunk, the ratio of senior to junior openings has worsened, and the first rung of the ladder — the routine work that used to be handed to a new graduate — is the part AI absorbed first.

But every piece of evidence in this report also points the same way about *who* is getting hired. Mercer's own data puts the assessed-employability gap between a top-tier and a third-tier college at about five percentage points. Zoho's founder says no job there requires a degree. Flipkart's early-careers page says all branches and all CGPAs. The NASSCOM analysts write that the question is no longer where you studied but what you can do.

That is not consolation. It is a description of a market that has quietly stopped rewarding the things you cannot change about yourself, and started rewarding the only thing you can: **what you have actually built, and whether you can explain it.**

E 6

A closing argument

Three observations that, taken together, make the case for acting this academic year rather than the next one.

One — the apprenticeship has to move

For thirty years, Indian engineering education and the Indian technology industry operated a working division of labour. Colleges taught theory. Employers taught application, in a graduate's first nine to twelve months, through routine work supervised by seniors. TCS's chief executive described that training period precisely — around nine months before a fresher is deployed on a project.

Both halves of that arrangement are now under pressure. The routine work that constituted the apprenticeship is what AI absorbed first. And 38% of engineering leaders report that AI tools have **reduced the direct mentoring juniors receive from seniors**. Employers are responding rationally: hire fewer juniors, hire more laterals who need no training, and raise the bar for the juniors they do hire.

If the industry is no longer able to run the apprenticeship, and the work still requires one, then it has to happen before employment. There is only one institution positioned to provide it. This is an inference rather than a measured finding, and we state it as such — but it follows directly from the evidence, and we have not found a serious argument against it.

Two — the gap is application, not content

It would be convenient to conclude that the syllabus is obsolete. The evidence does not support that. Salesforce — a major GCC employer — publishes a list of what it screens for that reads like a VTU course catalogue, and recommends college course notes as preparation. Target's June 2026 interview asked the OSI layers, DDL/DML/TCL and transaction management. Cisco asked about semaphores, deadlock and DNS. Micron asked about paging, thrashing and ACID.

Of the thirteen rows in the gap analysis in E 1, only **two** describe genuinely absent content. The other eleven describe subjects already taught, examined in a way the course does not practise. That is a far more tractable problem than curriculum replacement, and it is one a department can begin addressing with a weekly oral revision hour and a changed project rubric.

Three — the window is open now, and it is specific to this state

Three things are true at the same time, and will not all remain true indefinitely:

- **VTU has created the slot.** Circular 3216 mandates five offline skill labs before the seventh semester, beyond the curriculum, with the content left to the college. The regulatory permission already exists.
- **Karnataka holds the employers.** 875+ GCC units, roughly 30% of India's total; 32% of its GCC talent; a state policy targeting 500 new GCCs by 2029 with a fund explicitly for academia collaboration. **Over 90% of leading GCCs already work with universities.**
- **Those employers say the constraint is talent quality**, and cite graduate job-readiness at 42.6%. They are not short of candidates. They are short of candidates who can demonstrate what they can do.

A college that closes the application gap over the next two or three cohorts will be supplying exactly what the largest employer concentration in the country says it cannot find, from inside the state those employers are already in, under a framework its own university has already authorised.

The last word belongs to the industry, not to us

Rajesh Nambiar, President of NASSCOM, in November 2026, on graduates arriving with skills that are already obsolete: *"The level of transformations required in the curriculum has never been this significant."* And, on what that means for students now in their degree: *"From a fresher's point of view, it is a concern, and I will not sugarcoat it any differently."*

We would add only this. The same body of evidence that makes that statement true also shows the gap between a tier-one and a tier-three student's assessed capability to be about five percentage points — and shows employer after employer, from Zoho to Flipkart to Cisco, formally dropping the filters that used to stand between those students and the work.

The difficulty is real. So is the opening. Both of them are in front of your current second-year batch, and they will be decided by what that batch is asked to build between now and their seventh semester.

APPENDIX 1

The eligibility traps

More students are lost to administrative disqualification than to technical failure. Every item below is documented against a named company. Brief students on this list before applications open, not after.

Trap	Where it is documented
60% at every level, not on aggregate	**TCS** requires 60% at 10th, 12th, diploma, graduation and post-graduation separately. **Capgemini** warns some drives apply the cut-off to each level separately. **Accenture** applies it across Class 10, Class 12 and *all* graduation semesters.
No rounding	**Cognizant** — at least 60% with no rounding. **59.9% does not qualify.**
Pending backlogs	Disqualifying at **TCS, Cognizant, Accenture, Capgemini, HCLTech, LTIMindtree, Tech Mahindra, Cisco, Target, Arista, Chargebee**. **Wipro is the exception** — one backlog allowed at assessment, to be cleared with the 8th semester.
Extended education	**TCS** — the course must finish in the stipulated duration. A student who took an extra year to clear a subject is ineligible regardless of marks. This is the least-known trap on this list.
Education gap limits	**TCS** up to 24 months to the highest qualification. **Tech Mahindra** up to two years. **HCLTech and Capgemini** no more than **one year**. **Wipro** up to three years between 10th and the start of graduation but **none during graduation**.
How CGPA is computed	**Infosys** computes academics as the **simple arithmetic average across all semesters and all subjects, including electives, practicals and languages** — which is not how most universities compute CGPA, and can differ materially.
Branch restrictions	**Infosys SP/DSE** — CSE, IS/IT, Data Science, AI/ML, Cybersecurity, Software Engineering, ECE, EEE. **Wipro Turbo** — CS, IT or circuit branches at around 46 named institutes. **LTIMindtree** — CSE, IT, ECE, EEE. **Wipro Elite** excludes Fashion Technology, Textile Engineering and Agriculture & Food Technology. **SAP Labs** hires from core computing branches. **Target's apprenticeship** — CS, IT or circuit branches. **Navi** restricted one drive to CSE and MnC. **Cisco, Accenture, Capgemini, Tech Mahindra, Juspay and Flipkart are open to all branches.**
Campus-tier restriction	**Walmart Sparkplug** is officially limited to **35 selected colleges** (not named). **Wipro Turbo** to around 46 institutes. Most other programmes publish no campus list, which in practice means tier-1 first.
CGPA floors where documented	**Walmart** 7.0 OFFICIAL. **Oracle** 7.0 in one drive. **JPMorgan** 8.5 in one cycle. **Adobe** intern shortlists mostly above 7.5. **Arista** 70% or 7.0 across B.Tech, 10th and 12th. **NVIDIA** publishes no minimum; student reports cite 8.0+. **Zomato** gated one on-campus assessment at 8.0.

Trap	Where it is documented
Cool-off periods	**Wipro** — no re-application within three months of a previous selection process; re-registration 90 days after the last online assessment. **Flipkart** — a six-month cool-off is reported.
Experience counts against you	**Cognizant** — freshers only; prior work experience is not counted.
Mode of study	**TCS** — full-time only; part-time and correspondence excluded, though NIOS open schooling is accepted at 10th and 12th if graduation is full-time. **Wipro** — full-time degree only; distance accepted only for 10th and 12th.
Paid programmes	**HCLTech TechBee** carries a **fee of ₹1.40 lakh** for the IT Services track. It is a Class XII entry programme, not a graduate track. Both facts must be disclosed to students and parents.
Service agreements	**Wipro WILP** carries a **60-month service agreement**; leaving early triggers a pro-rata recovery. **Wipro Turbo** has a 12-month agreement of ₹75,000.

The briefing that prevents most of this

One hour, at the start of the sixth semester, walking the batch through this table with their own mark sheets in front of them. Students who discover in semester seven that a 59.4% in Class 12 or a one-year gap disqualifies them from half the drives cannot do anything about it. Students who discover it in semester six can at least plan around it — and can target the companies, listed throughout Part B, that do not apply these filters.

APPENDIX 2

Sources

Grouped by the claim they support. Official and press sources are listed first within each group. Candidate accounts are listed as a class rather than individually, since there are more than sixty of them; the company profiles in Part B name the platform and date for each.

Employability, placement and the Indian market

- Mercer | Mettl, *India's Graduate Skill Index 2025*, February 2025 — the 42.6% figure, tier-wise data and methodology. *No 2026 edition exists; do not cite a 2026 Mercer figure.*
- Wheebox/ETS with CII, AICTE and AIU, *India Skills Report 2026*, released 11–12 November 2025 — 70.15% for BE/BTech, 56.35% overall.
- AICTE placement and enrolment data, as reported by ThePrint (7 March 2025) and Careers360 (24 October 2025) — 47.7% UG placement 2019-20 to 2022-23; seat fill rates; Karnataka enrolment.
- Business Standard, 22 January 2025 — the 2025 placement recovery; entry-level IT services ₹3.6–4 lakh versus GCC ~₹6 lakh.
- Careers360, 17 November 2025 — IIT placements 2025-26; quant firms up 50%; consulting withdrawal.

AI and the entry-level job

- GitHub, *Octoverse 2026*, 28 October 2026 — agent-started pull requests over 40%; 32.0 million Indian developers.
- Stack Overflow, *Developer Survey 2025*, AI section — 84% usage, 33% trust, 46% distrust, 66% "almost right but not quite", 45.2% on debugging. *Figures independently re-verified for this report.*
- BCG X, *Rebuilding the Engineering Growth Ladder for an AI-First World*, 28 November 2025 — the four skills; 38% on reduced mentoring; 43% on critical thinking.
- World Economic Forum and PwC, *How AI is Changing Early Careers*, January 2026 — 36% versus 38% executive split; India excitement at 61%.
- NASSCOM, *Annual Strategic Review 2026*, 24 February 2026 — industry employment 59.5 lakh; net additions ~1.35 lakh.
- NASSCOM, July 2026, as reported — AI handling routine coding; 23% of professionals AI-native; the call for engineering judgement over basic coding lessons.
- Rajesh Nambiar, President NASSCOM, interviewed by Nikkei Asia 6 November 2026, **as reported by Elets CIO on 8 November 2026**. *We have not seen the Nikkei original; quotes are attributed accordingly.*
- Business Today, 6 August 2026, reporting the foundit Insights Tracker July 2026 — the "one in four entry-level IT jobs" quotation from Tarun Sinha; GCC share 8→11%; IT services 32→24%; six in ten postings requiring internships; AI roles at 4% of fresher hiring with 51% in AI operations.
- Business Standard, 16 July 2026, reporting 1 Finance Research on the AI pay premium — **20% at 0–3 years**. *We use this rather than the widely circulated 38% headline, which applies to 10+ years.*
- Business Today, 13 April 2026 and Business Standard, 12 April and 8 October 2026 — TCS fresher volumes and K Krithivasan's remarks. Business Today, 27–28 July 2025 — the TCS workforce reduction and the CEO's denial that AI caused it.

- People Matters, 16 April 2026 — Wipro 7,500 freshers, no FY27 target. Free Press Journal — Cognizant 25,000 for 2026, CFO Jatin Dalal.
- Fortune, 24 October 2025 — Cognizant CEO Ravi Kumar S on the broader, shorter pyramid.

GCCs and startups

- NASSCOM–Zinnov, *India GCC Landscape Report 2026 | GCC Value Orbit*, 6 May 2026 — 2,117 GCCs, 3,728 units, 2.36 million professionals, US$98.4bn, 1,200+ with AI capability, over 90% engaging universities.
- Dun & Bradstreet, *Decoding India's GCC Ecosystem — Bengaluru Edition 2026* — Karnataka shares and the state GCC policy. *Contains internal inconsistencies on national counts; used only for Karnataka shares.*
- Taggd with CII, *GCC Talent Lab Report 2026*, September 2026 — freshers 21%→16% of FY27 GCC hiring; 78% buy versus 22% build; talent quality as the leading constraint.
- Zinnov estimates reported via Nikkei Asia, October 2026 — 55% of GCC work facing AI displacement pressure; headcount per centre.
- foundit startup hiring data via CIOL, 15 May 2026; TeamLease EdTech Career Outlook, February 2026; Naukri JobSpeak, February and September 2026; PrepNPlaced *India Tech Hiring Report 2026*, October 2026 — the 4.3% entry-level share. *A one-month scrape of published postings, not a census.*

Company processes

- **Official:** TCS All India NQT page; Salesforce Futureforce engineering blog; Cisco Code with Cisco; Myntra RampUp listing; Flipkart Early Careers; Walmart Sparkplug listing; Target Apprentice — Technology; Meesho Open Campus on HackerEarth; Zoho Schools of Learning press release, 22 April 2025; Freshworks Software Academy FAQ; Goldman Sachs New Analyst postings; SAP iXp listings.
- **Press:** The Hans India, 13 January 2026 (HCLTech elite cadre, CPO Ram Sundararajan); The Week, 12 October 2025 and StartupPedia, 4 December 2025 (Zoho and Sridhar Vembu); HRKatha, 26 August 2025 (Flipkart early talent); Business Standard, People Matters, YourStory, Storyboard18 and Inc42 (Dream11 and Ola Krutrim).
- **Candidate-reported:** more than sixty individual accounts on GeeksforGeeks, LeetCode Discuss, Medium, Glassdoor, Taro, takeuforward, workat.tech and myuvce, together with the interview-preparation aggregators FACE Prep, Talentd, PrepInsta, PlacementPreparation.io and Design Gurus. Each is named with its platform and date in the relevant company profile.

What we deliberately excluded, and why

We excluded a large body of search-optimised "2026 interview guide" and "hiring trends" content that carries current datelines but has no named author, no named candidate and no traceable primary source — including one page quoting a US-dollar salary band for an India-based fresher role, and several whose round structures contradict their own related pages. We also excluded vendor listicles about which companies permit AI tools in interviews, since they are published by AI-interview-assistant vendors and are conflicted.

We have left the resulting gaps as gaps. Where this report says **not verified**, it means we looked.

APPENDIX 3

Where every question came from

This appendix exists so that nothing in Part D has to be taken on trust. Every question in this report marked **Asked at ...** traces to a specific, publicly readable account listed below. Open the link, find the question.

How to read this appendix, and what it does not claim

These are **candidate accounts** — individuals writing up their own interviews on public platforms. They are the only substantial public record of Indian fresher interviewing that exists, and they are genuinely specific, but each describes **one candidate, one cycle, one role**. We have given the date of each so the reader can judge staleness for themselves.

Questions in Part D marked *representative* are **ours**. They were written to match the documented pattern for that round and topic so that a department has enough material to teach and set assessments with. We have not attributed them to any company, and nobody should.

Candidate interview accounts, by company

Flipkart

Machine coding 90 min + 45-min review; COVID vaccination centre problem; bipartite graph, subarray-sum-k, top-k heap

geeksforgeeks.org/flipkart-interview-experience-for-sde-1off-campus-6-months-experienced/ (23 Feb 2022)
leetcode.com/discuss/interview-experience/1354371/flipkart-sde-1-bangalore-india

Swiggy

Shopping-cart and driver-assignment machine coding; Java/Golang-only assessment; JVM and Spring questioning; the 2021 loop with no machine coding

leetcode.com/discuss/interview-experience/7424272 (19 Dec 2025, upd. 3 Jan 2026)
leetcode.com/discuss/post/7642548/swiggy-sde-1-interview-experience-by-ano-4lnc/ (12 Mar 2026)
geeksforgeeks.org/swiggy-interview-experience-for-sde-1-off-campus-2021/ (6 May 2021)

Zomato / Eternal

The Tier-3 candidate who cleared; online-ordering system design, geo queries, pre-signed URLs, MongoDB left-prefix rule; the B+ tree round

leetcode.com/discuss/interview-experience/7407389 (Dec 2025)
leetcode.com/discuss/interview-experience/7046636

Razorpay

The 2.5-hour in-memory database build; the 90-minute photo-sharing LLD with MVC and Singleton

leetcode.com/discuss/interview-experience/1522135/razorpay-sde-1-banglore-sept-2021 (Sept 2021)
leetcode.com/discuss/interview-experience/5931375

Groww

Group-chat machine coding in a shared document; the tiered rate-limiter LLD; decode-ways DP; two-heap problem

leetcode.com/discuss/post/1645191/groww-sde-1-backend-bangalore

Meesho

Ride-sharing LLD with supplied signatures; SOLID and mutex questioning; graph route optimisation; the stipend and CTC figures

leetcode.com/discuss/interview-experience/7424523 (2025)
geeksforgeeks.org/meesho-interview-experience-for-sde-1-on-campus/ (7 Dec 2021)

PhonePe

The 2025 campus loop, code written on paper, digit DP and bitmask DP; the experienced-hire activity-tag machine coding and “Design Twitter”

leetcode.com/discuss/interview-experience/7087077 (2025 campus)
leetcode.com/discuss/interview-experience/6293258/ (Dec 2024)

CRED

150-minute machine coding with helpers supplied and unit tests required; “no UI/API”; tries and DSU; the contact-management LLD

leetcode.com/discuss/interview-experience/2657100 (3 Oct 2022)
leetcode.com/discuss/interview-question/1618127/cred-onsite-lld-contact-management-system (2021)

Uber

The multi-device controller OOP round; “design MS Excel”; base-conversion, Burst Balloons variant, capital/profit greedy

geeksforgeeks.org/uber-interview-experience-for-sde/ (11 Jan 2023)
geeksforgeeks.org/uber-interview-experience-for-sde-1-2022/ (2022)

Zepto

E-commerce LLD with coupon combinability; “design a DB schema for Uber”; the “prove the 15%” question; indexing and CAP

leetcode.com/discuss/interview-experience/5776849 (Sept 2024)
geeksforgeeks.org/zepto-interview-experience-for-sde-2-fte/ (6 Mar 2023, upd. 23 Jul 2025)

Ola

The notification-system LLD+HLD round *SDE-2, not fresher — labelled as such in Part B*

leetcode.com/discuss/post/5616645/ola-sde-2-bengaluru-july-2024-offer (Jul 2024)

Navi

Code written on paper; Asteroid Collision, Parallel Courses III, Sieve, in-place tree transform; the 30-MCQ screen; CSE/MnC branch restriction

geeksforgeeks.org/navi-interview-experience-for-sde-on-campus-2023/ (31 May 2023)
geeksforgeeks.org/navi-technologies-interview-experience-for-sde1-on-campus/ (1 Dec 2023)
geeksforgeeks.org/navi-interview-process-for-sde-1-off-campus/ (2 Aug 2023)

Target India

The June 2026 interview: four OOP pillars, seven OSI layers, DDL/DML/TCL, transactions, bubble sort — the most current account in this report

geeksforgeeks.org/interview-experiences/target-india-apprenticeship-program-2026-interview-experience/ (interview 13 Jun 2026, pub. 7 Jul 2026)
geeksforgeeks.org/target-corporation-india-interview-experience-on-campus/ (8 Apr 2020)

Cisco

42-question assessment; FTP active/passive, DHCP, DNS, semaphores, deadlock, circular linked list

geeksforgeeks.org/interview-experiences/cisco-interview-experience-for-summer-intern-2025/ (interview 19 Sep 2025)

Walmart Global Tech

Three-stage assessment funnel; DSU, Tarjan's articulation points, divisor queries at 10^8

geeksforgeeks.org/interview-experiences/walmart-sparkplug-interview-experience-for-sde-internship-2025 (cycle Oct 2024)

Salesforce

LRU cache, whiteboarded system flow, abstract class vs interface, static/dynamic binding, producer–consumer, DBMS locking

geeksforgeeks.org/interview-experiences/salesforce-interview-experience-for-sde-internship-on-campus/ (8 Sep 2020)
salesforce.com/blog/futureforce-india-software-engineer/ OFFICIAL (5 Mar 2024, mod. 1 Oct 2026)

Oracle

Unique BSTs in the assessment; normalization + self-join; COUNT over a UNION of joins; dense vs sparse indexing; Linux boot process; the 8-balls and daughters'-ages puzzles

geeksforgeeks.org/interview-experiences/oracle-interview-experience-application-developeron-campus-12th-august-2024/ (drive 12 Aug 2024)

Micron

Thrashing, paging algorithms, semaphores vs mutex, zombie processes, RAID, ACID, Nth-highest salary; the 15-minute OOP block

geeksforgeeks.org/interview-experiences/micron-technology-interview-experience-on-campus/ (campus 29 Aug 2021)

Intuit

Product-of-array-except-self pushed to O(N) then O(1); paging, process states, FCFS and Round Robin, normalization

geeksforgeeks.org/interview-experiences/intuit-interview-experience-for-internship-on-campus/ (13 Sep 2023)

Qualcomm

The three-section 60-MCQ test; binary↔hex conversion; BST validation, doubly linked lists, hashing

geeksforgeeks.org/interview-experiences/qualcomm-interview-experience-on-campus-for-internship/ (22 Sep 2020)

Arista Networks

The 50-minute URL-shortener system-design round; stack with O(1) minimum; binary linked list to decimal; "implement an STL vector in C"; TCP/IP, OSI, routing, switching, SDN

geeksforgeeks.org/interview-experiences/arista-networks-interview-experience-for-software-engineer-on-campus/ (3 Feb 2022)
geeksforgeeks.org/interview-experiences/arista-networks-recruitment-process/ (upd. 20 Sep 2025, compiled)

Goldman Sachs

Convex hull, Trie design, grid path DP, BFS minimum operations, minimum platforms, postfix evaluation; probability and statistics content; the CoderPad round

geeksforgeeks.org/interview-experiences/goldman-sachs-interview-experience-for-analyst-position/ (22 Sep 2020)
geeksforgeeks.org/interview-experiences/goldman-sachs-interview-experience-set-22-campus-analyst-profile/ (20 Jan 2017 — **very stale**)

JPMorgan Chase

The HireVue elimination round; merge intervals and substring counting; Code for Good 24-hour hackathon mechanics

geeksforgeeks.org/jp-morgan-chase-co-interview-experience-code-for-good-hackathon-2024 (10 Sep 2024)
geeksforgeeks.org/interview-experiences/jpmorgan-chase-code-for-good-interview-experience-for-software-engineer-fte/ (27 Jul 2021)

Adobe

Bayes' theorem and linear-algebra MCQs; the compiled process description listing LLD and design patterns in Technical 3

geeksforgeeks.org/interview-experiences/adobe-recruitment-process/ (upd. 28 Jul 2026, compiled)
geeksforgeeks.org/?p=897269 (25 Oct 2022)

Lowe's India

OS and DBMS MCQs with time complexity; the equal-halves substring and left-subtree-sum problems

geeksforgeeks.org/lowes-india-interview-experience-for-sde-i-on-campus/ (25 Jul 2022)

Rakuten

The 2048 tile-puzzle implementation; Stock Span; "design a browser backend"; DBMS locking, Java internals, concurrency

geeksforgeeks.org/interview-experiences/rakuten-interview-experience-sde-intern-pool-campus/ (8 Jul 2020)

BrowserStack

Both machine-coding rounds (log watcher; stateless browser service); the 1 TB file with 8 GB RAM question; HTTP vs HTTPS, status codes, User-Agent

geeksforgeeks.org/interview-experiences/browserstack-interview-experience-for-freshers/ (5 Mar 2020, upd. 12 Jul 2025)

Zeta (Directi)

Library-management LLD then Ludo design-code-debug; DSU; “what happens when you enter a URL”; TCP, UDP, sockets; HTTP vs HTTPS

geeksforgeeks.org/interview-experiences/zeta-directi-interview-experience-for-sde-intern/ (10 Feb 2025)

Juspay

The Hiring Challenge structure; LRU cache, JSON indentation, Stock Span; functional programming for backend roles

geeksforgeeks.org/interview-experiences/juspay-recruitment-process/ (20 Nov 2023, upd. 4 Aug 2026)
geeksforgeeks.org/juspay-hiring-challenge/ (2 Mar 2022, upd. 18 Jun 2024)
talentd.in/articles/juspay-hiring-challenge (checked 30 Oct 2026)

InMobi

The open-ended hospital appointment system; next-smaller-element reduced to constant space; maximum visible points

geeksforgeeks.org/interview-experiences/inmobi-interview-experience-on-campus-2021/ (28 Dec 2021)

Chargebee

Maximum product subarray, coin change, delete without head pointer, flattening a linked list, palindromic partitioning; the 60%/6.5 criteria

geeksforgeeks.org/interview-experiences/chargebee-recruitment-process/ (6 Sep 2022, upd. 23 Jul 2025)

Innovaccer

The week-long entry-management take-home; Nth-highest salary with and without a nested query *2019–20 — flagged as very stale in Part B*

geeksforgeeks.org/innovaccer-interview-experience-sde-intern/ (26 Apr 2020)

Freshworks, Sprinklr, Darwinbox, HackerRank, Hasura, Postman

Round structures and problems as cited in B 5

designgurus.io/answers/detail/what-is-the-freshworks-interview-process-like-round-by-round (undated)
takeuforward.org/interviews/sprinklr-interview-experience-set-1 (undated)
medium.com/@lokeshwar777/my-recent-on-campus-interview-experience-with-darwinbox-cedf2872583a (7 Sep 2024)
anubhav-gupta.medium.com/hackerrank-sde-intern-interview-experience-437a06d66bcb (events Oct–Dec 2023)
github.com/kamleshchandnani/awesome-interview-processes (community-maintained, undated)
hirestepx.com/blog/postman-interview-process-sde-india-2026 (19 Jul 2026)

Zoho

The five-to-seven round structure; the C/C++-only advanced programming round; the booking-system problem; marking on quality and modularity

faceprep.in/article/zoho-test-pattern-in-detail/ (7 May 2026)
faceprep.in/article/zoho-off-campus-hiring-2026-eligibility-and-process/ (12 Aug 2026)
placementpreparation.io/zoho/recruitment-process/ (undated)

IT services — TCS, Infosys, Wipro, Cognizant, HCLTech, Accenture, Capgemini, LTIMindtree, Tech Mahindra

Assessment patterns, topic lists, Automata Fix bug types, pseudocode components, essay and story-writing rounds, track CTC bands

Official: tcs.com/careers/india/tcs-all-india-nqt-hiring
Aggregators: faceprep.in (TCS, Infosys, Cognizant, HCLTech, Accenture, Capgemini, LTIMindtree, Tech Mahindra guides, May–Aug 2026) · talentd.in/articles/wipro-elite-nth (30 Oct 2026) · talentd.in/articles/cognizant-genc (30 Oct 2026) · prepinsta.com/infosys-sp-and-dse/ · prepinsta.com/wipro/ (mod. 17 Feb 2025)

The practice and assessment platforms named in this report

Platforms on which real assessments in this study were actually administered, per candidate accounts: **HackerRank** (TCS, JPMorgan, Salesforce CodePair, Arista, CRED, Freshworks, Adobe), **HackerEarth** (Meesho Open Campus, Sprinklr, Target), **DoSelect** (Navi), **CodeSignal** (PhonePe, experienced track), **CoderPad** (Goldman Sachs), **AMCAT and MeritTrac** (Cognizant), **SAB and Cocubes** (Capgemini), **Superset** (Wipro), **Unstop** (Myntra RampUp, Walmart Sparkplug), **TCS iON** test centres, and **Infosys** test centres.

Platforms for student practice, named for the college's own assessment programme in E 3: **leetcode.com** · **neetcode.io** · **hackerrank.com** · **hackerearth.com** · **codeforces.com** · **codechef.com** · **atcoder.jp** · **interviewbit.com** · **geeksforgeeks.org** · **takeuforward.org** · **workat.tech/machine-coding/practice** (the one platform in this list built specifically for the machine-coding round) · **unstop.com** for open programmes.

How to verify any claim in this report

A question in Part D: find the company in the list above and open the linked account. If the tag says *representative*, it is ours and there is nothing to find.

A process description in Part B: the label on the paragraph says whether it is official, press or candidate-reported, and the company's sources are listed above or in Appendix 2.

A market figure in Part A: Appendix 2 names the publishing organisation and date for every one.

Anything that says NOT VERIFIED: we looked and could not find a credible source. If you can, we would genuinely like to know — the contact details are at the end of this report, and a corrected edition is better than a confident one.

APPENDIX 4

What this report does not know

A report that claims to describe hiring across an entire industry should be explicit about the edges of its own evidence. These are the limitations we are aware of. We would rather state them than have a reader find them.

1 · The public record on interview processes is thin, and skewed

Across 35 employers we found genuinely official process documentation for a handful. Everything else rests on candidate accounts, which are **self-selected**: people who were offered a job write up their experience more often than people who were not, and people who faced an unusual round write about it more often than people who faced a routine one. The round structures in Part B are therefore better evidence of **what can happen** than of what happens on average.

2 · Much of the evidence is older than we would like

We have 2025–26 accounts for Swiggy, Zomato/Eternal, Meesho, PhonePe, Target, Cisco, Zeta and Zoho. For Flipkart, Razorpay, CRED, Uber, BrowserStack, Goldman Sachs, Micron, Qualcomm, Rakuten, Lowe's and Innovaccer the best available accounts are 2020–2023, and each is flagged in place. Interview processes change every cycle. **Treat anything pre-2025 as evidence of style, not of current practice.**

3 · Compensation is the weakest part of the record

Apart from the TCS NQT offer table, Walmart's published internship stipend, Myntra's stipend, Atlan's stipend and the Juspay figures, essentially no fresher CTC in this report is officially sourced. We have therefore printed very few salary numbers and labelled every one. **A placement cell's own offer letters are the only reliable source, and they are better than anything published.**

4 · Several widely circulated figures were deliberately excluded

There is no Mercer-Mettl Graduate Skill Index for 2026; the latest is the 2025 edition with 2024 data, and anyone citing a “2026 Mercer figure” is citing something that does not exist. There is **no authoritative national dataset on median versus top engineering campus packages**, so we describe the widening spread through indirect evidence rather than inventing a ratio. The “only 20% of engineers are employable” line that still circulates traces to Aspiring Minds reports of 2016–2019 and has no 2025 or 2026 equivalent. A “56% AI salary premium” and a “20–25% campus AI premium” both circulate with no traceable primary study; we used the 1 Finance figure of 20% at 0–3 years instead, which has a stated method and a sample of 950 postings.

5 · Two headline claims rest on secondary reporting

The Rajesh Nambiar quotations in Part A come from an Elets CIO report of a Nikkei Asia interview; **we have not seen the Nikkei original** and have attributed them accordingly throughout. The Zinnov figures on GCC work exposed to AI displacement likewise reach us through secondary reporting. Both are flagged where they appear.

6 · One tracker figure is widely quoted and deserves a caveat

The finding that entry-level roles are 4.3% of live tech postings, with roughly twelve senior openings per entry-level one, comes from a **one-month scrape of indexed LinkedIn and Naukri feeds** — not a census. Its own authors note it skews towards companies that advertise openly and therefore understates walk-in and agency hiring. We use it as one tracker's finding, which is what it is.

7 · We could not answer the AI-tools question at all

No employer in this study publishes a policy on whether generative AI tools may be used in its assessments. We searched specifically and found nothing official from any Indian company. The only indirect signal is Infosys running its October 2026 Specialist Programmer coding assessment in person at its own centres. **This is a real gap with practical consequences for every college**, and the only way to close it is for placement cells to ask their campus-recruitment contacts directly.

8 · What is argument rather than evidence

Three claims in this report are **inferences** and are labelled as such where they appear: that the apprenticeship which used to happen in a graduate's first year must now happen before employment; that the gap between tier-one and tier-three outcomes is largely a gap in demonstrated rather than actual capability; and that a structured skill-lab programme is the right institutional response. Each follows from the evidence presented, and each is a judgement a reader is entitled to disagree with.

On how this report was produced

This report was researched and written with substantial use of AI tooling, under human direction and review — which is, not incidentally, the working method it recommends to students in Part D 8. That is precisely why Appendix 3 exists. Every question attributed to a company carries a public link; every market figure carries a named publisher and date; and everything we could not verify is marked as unverified rather than smoothed over.

We would rather be checked than believed. If you find an error, write to us and the next edition will carry the correction.

ABOUT

Algorithms365

We are a Bengaluru software-skilling company whose entire work sits in the gap this report describes: the distance between what a campus produces and what a corporate hires.

What we do

Algorithms365 Technologies LLP trains engineers to be hired — and, increasingly, works with the institutions that produce them. Our teaching follows one guiding sequence, arrived at from watching what actually converts into offers: **data structures, then low-level design, then AI and communication, then building real software.** Each stage exists because a specific hiring round demands it, and the order matters more than the topic list.

Where we work	What is there
YouTube **youtube.com/@algorithms365**	Over **200 hours of free content** across more than 600 videos — DSA, Java, Python, full stack, operating systems, networking, system design, aptitude and interview preparation, taught in **English and Kannada**. Roughly 92,000 subscribers. Free, with no registration.
Learning platform **skills.algorithms365.com**	Over **300 hours** of structured programmes with assessments and progress tracking.
Offline centre **HSR Layout, Bengaluru**	In-person upskilling, currently running intensive placement preparation for the **2026 graduating batch**.
Instagram **@algorithms365_kannada**	Short-form teaching and placement updates.

Mahesh C. Arali — Founder

Mahesh founded Algorithms365 to work on a problem he kept meeting from both sides: capable engineering graduates who could not demonstrate what they knew, and employers who could not find people who could. His argument, which runs through this report, is that the deficiency is rarely in the student and rarely in the syllabus — it is in the absence of any structured place where a student is made to **apply** what they have been taught, under pressure, in front of someone who asks why.

Most of what Algorithms365 teaches is published free on YouTube for exactly that reason: the knowledge is not the scarce resource. What is scarce is practice, feedback and the chance to build something real — and that is what the offline centre, the learning platform and our work with colleges exist to provide.

How we work with colleges

We are open to working with institutions in any of four ways, remotely or at our Bengaluru office:

- **Skill labs** attached to the semesters where the matching theory is taught — the six-lab, 300-hour model set out in E 2 and detailed in our companion roadmap, designed against VTU circular 3216.
- **Campus placement preparation** built around the nine rounds in Part C, rather than around aptitude alone.

- **Curriculum and structure consulting** — mapping an existing programme against the gap analysis in E 1 and advising on what to change, including the assessment programme in E 3.
- **Faculty development programmes**, so that the teaching capability stays in the institution rather than depending on an external vendor indefinitely. We think this last one matters most, and we would rather a college eventually needed us less.

> ### Hold us to E 3
>
> Section E 3 argues that any skilling partner who cannot show measured improvement on an instrument the college controls is not delivering what the college is paying for. **That applies to us.** We would encourage any institution working with us to baseline its cohort before we start, measure it on its own terms, and judge the engagement on the distribution of the result.

Why we think the next few years matter

Most commentary on AI and software employment stops at the disruption. We think that is only the first half of the story, and the less interesting half.

For the whole history of the industry, software has been expensive to build. That cost decided what got built: large markets, well-funded customers, problems worth a team of engineers for a year. An enormous number of real problems — in Indian agriculture, in local government, in small manufacturing, in schools, in healthcare at the district level — were never addressed, not because they were unimportant or unsolvable, but because nobody could justify the engineering cost.

That constraint is lifting. For the first time, software is becoming genuinely affordable to build. When the disruption settles, the reasonable expectation is not less software development but far more of it — aimed at problems that were previously beneath the economic threshold, built by smaller teams, in more places.

Which is why this report, for all its uncomfortable numbers, is not a pessimistic document. The engineers who will build that wave are, right now, sitting in second-year classrooms across Karnataka. The question this report puts to their institutions is a narrow and answerable one: **between now and their seventh semester, what will they actually be asked to build?**

> ### Talk to us
>
> **Mahesh C. Arali** — Founder, Algorithms365 Technologies LLP
> **mahesh@algorithms365.com** · **+91 99452 75585**
> www.algorithms365.com · skills.algorithms365.com · youtube.com/@algorithms365
> HSR Layout, Bengaluru — further contact numbers are on the website.
>
> We are glad to discuss what this report means for your institution, to present it to your faculty or management, or simply to be told where we have got it wrong.

algorithms

The gap is not what students know. It is what they have built.

Algorithms365 works with engineering colleges to attach structured skill labs to the VTU 2025 scheme — turning taught theory into demonstrated, hireable skill. We would be glad to discuss what this report means for your institution.

Mahesh C. Arali · Founder & CEO

mahesh@algorithms365.com · +91 99452 75585

www.algorithms365.com · skills.algorithms365.com

youtube.com/@algorithms365 · @algorithms365_kannada

Algorithms365 Technologies LLP · HSR Layout, Bengaluru · November 2026